
Instalación y Configuración de Sistemas Gestores de Bases de Datos



ASIGNATURA: BASES DE DATOS 2

PRÁCTICA 1

CURSO: 2025/26

COORDINADORA: Vázquez Martín, Lucía (NIP 871886)
Zhou Chen, Luna (NIP 812103)

Índice

Índice.....	2
1. Introducción.....	4
2. Configuración del entorno de trabajo.....	4
2.1. Despliegue de la máquina virtual.....	4
2.2. Acceso administrativo vía SSH.....	4
2.3. Instalación del motor de contenedores (Docker).....	5
2.4. Instalación de DBeaver.....	6
3. PostgreSQL.....	9
3.1. Definición del servicio y control de versiones.....	9
3.2. Configuración de seguridad y gestión de roles.....	10
3.3. Verificación de políticas de seguridad.....	11
3.4. Estrategia de copia de seguridad.....	12
3.5. Generación de datos y pruebas.....	13
4. Oracle XE.....	14
4.1. Definición del servicio y control de versiones.....	14
4.2. Configuración de seguridad y gestión de roles.....	15
4.3. Verificación de políticas de seguridad.....	19
4.4. Estrategia de copia de seguridad.....	20
4.5. Generación de datos y pruebas.....	23
5. Apache Cassandra.....	24
5.1. Definición del servicio y control de versiones.....	24
5.2. Configuración de seguridad y roles.....	25
5.3. Verificación de políticas de seguridad.....	26
5.4. Estrategia de copia de seguridad.....	28
5.5. Generación de datos y pruebas.....	29
6. Apache HBase.....	30
6.1. Definición del servicio y control de versiones.....	30
6.2. Creación de estructura de datos.....	31
6.3. Seguridad en Apache HBase.....	33
6.4. Estrategia de copia de seguridad: Snapshots en HBase.....	36
6.5. Generación de datos y pruebas.....	37
7. IBM Db2.....	38
7.1. Definición del servicio y control de versiones.....	38
7.2. Creación y configuración inicial de la base de datos.....	41
7.3. Configuración de seguridad y gestión de roles.....	42
7.4. Estrategia de copia de seguridad.....	44
7.5. Generación de datos y pruebas.....	45
8. Configuración de acceso remoto a los SGBD.....	49
8.1. PostgreSQL.....	51
8.2. Oracle XE.....	54
8.3. Apache Cassandra.....	56
8.4. Apache HBase.....	59

8.5. IBM Db2.....	61
9. Comentarios acerca de las licencias.....	64
9.1. PostgreSQL.....	64
9.2. Oracle XE.....	65
9.3. Apache Cassandra y Apache HBase.....	65
9.4. IBM Db2.....	65
10. Dificultades encontradas y soluciones.....	66
10.1. PostgreSQL: Error en conexión remota inicial desde DBeaver.....	66
10.2. Oracle XE: Conflicto conexión remota con imagen “slim”	67
10.3. Apache HBase: herramientas de acceso y visualización.....	67
10.4. Complejidades en la configuración de red.....	69
11. Conclusiones y comparativa.....	71
12. Organización del trabajo.....	72
13. Bibliografía.....	73
14. Anexo. Scripts de despliegue y configuración.....	74
14.1. Anexo I. PostgreSQL.....	74
14.2. Anexo II. Oracle.....	76
14.3. Anexo III. Apache Cassandra.....	79
14.4. Anexo IV. Apache HBase.....	80
14.5. Anexo V. IBM Db2.....	82

1. Introducción

En esta práctica se ha llevado a cabo la instalación y configuración de diversos sistemas gestores de bases de datos, tanto relacionales como NoSQL: PostgreSQL, Oracle Database Express Edition, Apache Cassandra, Apache HBase e IBM Db2.

El objetivo es familiarizarse con el despliegue, la configuración de seguridad, la creación de usuarios y la definición de esquemas de datos, así como comprobar su funcionamiento mediante la inserción y consulta de datos de prueba.

2. Configuración del entorno de trabajo

2.1. Despliegue de la máquina virtual

Se ha optado por utilizar una máquina virtual basada en **TurnKey Linux Core** (basada en Debian) debido a su ligereza y optimización para entornos de servidor. La virtualización se ha realizado mediante **VirtualBox**, asignando los siguientes recursos para garantizar el rendimiento de los múltiples contenedores:

- **Memoria RAM:** 4096 MB.
- **Procesadores:** 2 vCPU.
- **Red:** Configuración NAT con reenvío de puertos.

Es crucial destacar la configuración de red. Para permitir el acceso desde el sistema anfitrión (Windows) a los servicios desplegados en la VM, se han configurado reglas de reenvío de puertos (Port Forwarding), exponiendo tanto el puerto SSH (2222) como los puertos de los distintos SGBD, como el 5432 para PostgreSQL.

Nombre	Protocolo	IP anfitrión	Puerto anfitrión	IP invitado	Puerto invitado
Cassandra	TCP		9042		9042
Oracle	TCP		1521		1521
Postgres	TCP		5432		5432
ssh	TCP	127.0.0.1	2222		22
HBase	TCP		16010		16010
DB2	TCP		50000		50000

Figura 1. Configuración de reenvío de puertos (Port Forwarding) en VirtualBox

2.2. Acceso administrativo vía SSH

Con el objetivo de agilizar la administración del sistema y facilitar la inserción de comandos complejos mediante el portapapeles del sistema anfitrión se ha habilitado la conexión mediante el protocolo SSH.

Dado que la configuración de red y el reenvío de puertos se establecieron correctamente durante la creación de la máquina virtual, la conexión se realizó de manera directa desde la terminal de Windows (PowerShell) mediante el siguiente comando:

```
ssh -p 2222 root@localhost
```

Tras la autenticación con la contraseña de root se obtuvo acceso inmediato, habilitando la administración remota con privilegios de superusuario.

2.3. Instalación del motor de contenedores (Docker)

Para la gestión de los SGBD se ha optado por el uso de **Docker**, lo que permite aislar las dependencias de cada base de datos y mantener limpio el sistema operativo base. Al disponer de privilegios de root en la máquina Turnkey se procedió a la instalación ejecutando directamente los siguientes pasos para asegurar la versión más reciente:

1. Actualización de repositorios e instalación de prerequisites

```
apt update  
apt install ca-certificates curl gnupg lsb-release
```

2. Configuración del repositorio oficial de Docker

Se añadió la clave GPG oficial y se configuró el repositorio estable para la arquitectura de Debian Bookworm, asegurando así la integridad de los paquetes descargados.

3. Instalación de los binarios

Se procedió a la instalación del motor y las herramientas de cliente:

```
apt install docker-ce docker-ce-cli containerd.io
```

4. Instalación de Docker Compose

Para facilitar la orquestación de contenedores mediante ficheros YAML se descargó e instaló el binario de Docker Compose en /usr/local/bin, otorgándole permisos de ejecución:

```
curl -L  
"https://github.com/docker/compose/releases/latest/download/docker-compose-$(un  
ame -s)-$(uname -m)" -o /usr/local/bin/docker-compose  
  
chmod +x /usr/local/bin/docker-compose
```

5. Verificación

Se comprobó la correcta instalación mediante los comandos `docker --version` y `docker-compose --version`, obteniendo las versiones v29.2.1 y v5.0.2, respectivamente.

2.4. Instalación de DBeaver

Con el objetivo de disponer de una herramienta gráfica para la administración y prueba de los distintos sistemas gestores de bases de datos instalados durante la práctica, se procedió a la instalación de **DBeaver Enterprise Edition 25.3** en el sistema anfitrión (Windows).

DBeaver es un cliente multiplataforma que soporta múltiples motores relacionales y NoSQL, lo que lo convierte en una herramienta adecuada para trabajar de forma unificada con distintos SGBDs.

En primer lugar, se accedió al **sitio oficial de DBeaver** y se descargó el instalador correspondiente a la versión: DBeaver Enterprise Edition 25.3. Se optó por la edición Enterprise debido a su compatibilidad ampliada con distintos motores y herramientas avanzadas de administración.

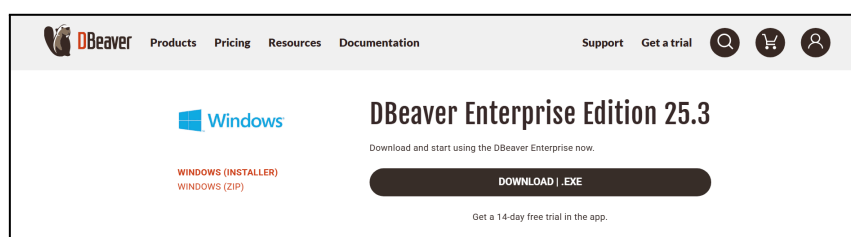


Figura 2. Sitio oficial de DBeaver

Dado que la Universidad de Zaragoza tiene un convenio con la empresa propietaria de la aplicación, se solicitó una licencia académica utilizando el correo institucional de la universidad (@unizar.es).

El proceso requirió:

1. Registro de una cuenta en la plataforma oficial de DBeaver.
2. Confirmación del correo electrónico institucional.
3. Solicitud de la licencia académica.

Una vez validada la cuenta, se recibió la licencia correspondiente para su activación en la aplicación.

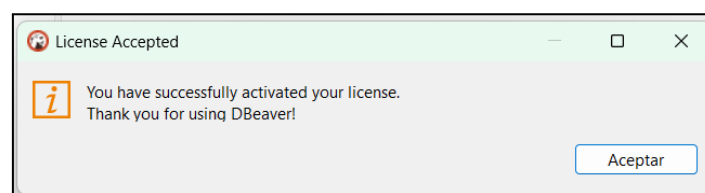
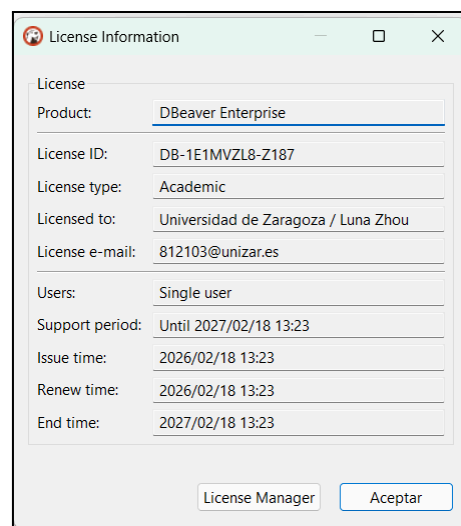
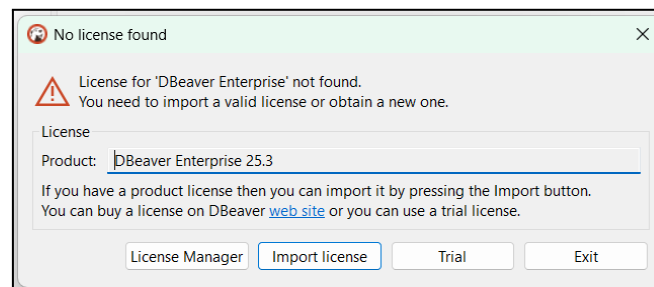
License ID:	DB-1E1MVZ18-Z187
License Type:	Academic
Maintenance Period:	1 year
Maximum users:	1
Issue Time:	Feb 18, 2026
Expiration Time:	Feb 18, 2027
License Status:	Valid

Figura 3. Información sobre la licencia académica adquirida

Luego, se ejecutó el instalador descargado y se siguió el asistente de instalación paso a paso. El procedimiento consistió en:

1. Aceptar los términos de licencia.
2. Seleccionar la carpeta de instalación.
3. Elegir los componentes a instalar.

En la selección de componentes se mantuvieron las opciones por defecto recomendadas por el asistente, dado que cubrían los drivers necesarios para los SGBD utilizados en la práctica. Una vez finalizada la instalación, se ejecutó la aplicación. En el primer inicio, el sistema solicitó la activación de la licencia. Se introdujo la licencia académica previamente obtenida, quedando activada correctamente la versión Enterprise.



Figuras 4, 5 y 6. Proceso de activación de la licencia para DBeaver

Tras ello, se accedió a la pantalla principal de DBeaver, desde la cual es posible crear nuevas conexiones, gestionar drivers y administrar bases de datos.

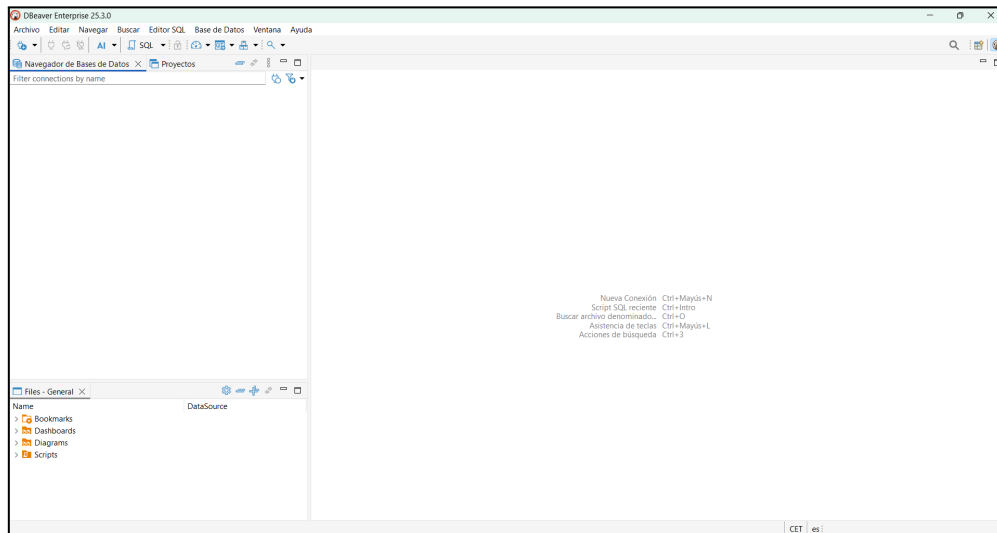


Figura 7. Pantalla de inicio de DBeaver

La creación de una nueva conexión a cualquier sistema gestor de bases de datos se realiza mediante el asistente integrado de DBeaver. Desde la pantalla principal, se debe seleccionar la opción **Base de Datos** → **Nueva Conexión**, o bien utilizar el icono correspondiente en la barra de herramientas. A continuación, el asistente solicita la selección del motor de base de datos con el que se desea trabajar (por ejemplo, PostgreSQL, IBM Db2, etc.).

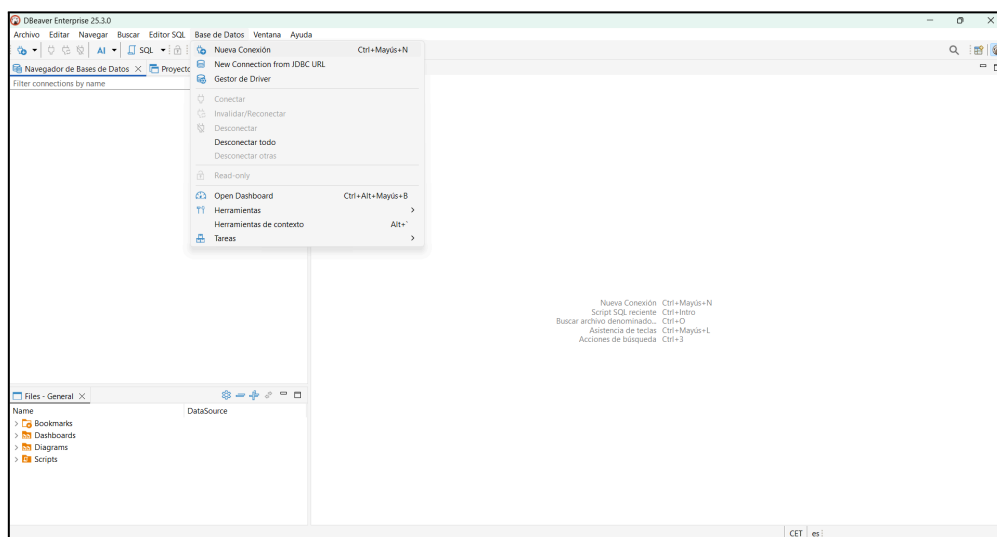


Figura 8. Proceso de creación de una nueva conexión en DBeaver

Tras seleccionar el motor, se accede al cuadro de configuración de conexión, donde deben completarse los siguientes parámetros fundamentales:

- **Host:** dirección del servidor.
- **Puerto:** puerto de escucha del servicio (según el SGBD correspondiente).
- **Nombre de la base de datos.**
- **Usuario y contraseña** previamente definidos en el sistema gestor.
- Opcionalmente, parámetros avanzados o propiedades adicionales del driver.

Una vez introducidos los datos, es recomendable utilizar la opción **Test Connection** para verificar que la conexión se establece correctamente y que las credenciales son válidas. Si la

prueba es satisfactoria, se finaliza el asistente, quedando registrada la conexión en el panel lateral izquierdo de la aplicación. A partir de ese momento, la base de datos puede expandirse para visualizar esquemas, tablas y demás objetos, así como ejecutar consultas SQL en el editor integrado.

En los apartados específicos dedicados a cada SGBD se detallarán únicamente los valores concretos empleados en cada conexión, dado que el procedimiento general descrito en esta sección es común para todos ellos.

3. PostgreSQL

3.1. Definición del servicio y control de versiones

Para garantizar la reproducibilidad y el control de versiones exigido en el enunciado, el despliegue se ha realizado mediante **Docker Compose**. Se ha seleccionado la imagen oficial **postgres:15.2** (en lugar de la etiqueta **latest**) para asegurar que el entorno de desarrollo sea idéntico al de producción y evitar actualizaciones automáticas no controladas.

A continuación, se muestra el contenido del fichero `docker-compose.yml` para PostgreSQL:

```
version: '3.8'
services:
  postgres:
    image: postgres:15
    container_name: postgres-practica1
    restart: always
    environment:
      POSTGRES_USER: alumno          # Superusuario inicial
      POSTGRES_PASSWORD: alumno123
      POSTGRES_DB: practica1_db
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data # Persistencia de datos

volumes:
  pg_data:
```

Justificación de la configuración:

- **volumes:** Se ha definido un volumen nombrado (`pg_data`) montado en la ruta `/var/lib/postgresql/data`. Esto asegura que la información almacenada en la base de datos sobreviva al reinicio o eliminación del contenedor.
- **environment:** Se han establecido las variables de entorno para inicializar automáticamente la base de datos `practica1_db` y el usuario administrador `alumno`.

El servicio se desplegó ejecutando:

```
docker-compose up -d
```

3.2. Configuración de seguridad y gestión de roles

Tras esto, se ha configurado un esquema de seguridad basado en el principio de mínimo privilegio. Para ello, se accedió al contenedor mediante el cliente `psql`:

```
docker exec -it postgres-practical1 psql -U alumno -d practical1_db
```

Dentro de la consola SQL, se verificó que el usuario **alumno** posee atributos de **superusuario**, tal y como se muestra a continuación:

```
root@core ~/practical1/postgres# docker exec -it postgres-practical1 psql -U
alumno -d practical1_db
psql (15.15 (Debian 15.15-1.pgdg13+1))
Type "help" for help.

practical1_db=# \du
                                List of roles
Role name |                               Attributes                               | Member
of
-----+-----+-----
alumno    | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
```

Además, siguiendo los requisitos de la práctica, se ha evitado el uso del superusuario (**alumno**) para operaciones rutinarias. En su lugar, se ha implementado un esquema de seguridad basado en roles (RBAC):

1. **Creación de roles:** Se han definido los roles **rol_lectura**, con permisos CONNECT y SELECT, y **rol_escritura**, que hereda de lectura y añade INSERT, UPDATE y DELETE.
2. **Asignación de usuarios:** Se han creado los usuarios **consultor** (asignado al rol de lectura) y **editor** (asignado al rol de escritura).

```
-- 1. Creación de roles
CREATE ROLE rol_lectura;
CREATE ROLE rol_escritura;

-- 2. Asignación de permisos a los roles --> permisos de Lectura (base)
GRANT CONNECT ON DATABASE practica1_db TO rol_lectura;
GRANT USAGE ON SCHEMA public TO rol_lectura;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rol_lectura;

-- Permisos de escritura (Hereda de lectura y añade modificaciones)
GRANT rol_lectura TO rol_escritura;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO rol_escritura;
```

```
-- Permiso para usar contadores automáticos (SERIAL/SEQUENCES)
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO rol_escritura;

-- 3. Creación y asignación de usuarios
-- consultor solo lectura, mientras que editor tanto lectura como escritura
CREATE USER consultor WITH PASSWORD 'consultor123';
GRANT rol_lectura TO consultor;

CREATE USER editor WITH PASSWORD 'editor123';
GRANT rol_escritura TO editor;
```

3.3. Verificación de políticas de seguridad

Para validar que la política de permisos quedaba correctamente aplicada, se creó una tabla de prueba y se insertaron datos iniciales como usuario administrador:

```
practica1_db=# CREATE TABLE pruebas_final (
  id SERIAL PRIMARY KEY,
  dato TEXT NOT NULL
);
CREATE TABLE

practica1_db=# INSERT INTO pruebas_final (dato) VALUES ('Dato visible 1'),
('Dato visible 2');
INSERT 0 2

practica1_db=# SELECT * FROM pruebas_final ORDER BY id;
 id |      dato
----+-----
  1 | Dato visible 1
  2 | Dato visible 2
(2 rows)
```

Durante la verificación se tuvo en cuenta un comportamiento relevante: GRANT ... ON ALL TABLES IN SCHEMA public **solo aplica a las tablas existentes en el momento del GRANT**. Por tanto, si los permisos “globales” se conceden antes de crear nuevas tablas, estas no quedan cubiertas automáticamente. Para asegurar acceso a la tabla recién creada, se concedieron permisos explícitos:

```
practica1_db=# GRANT SELECT ON TABLE pruebas_final TO rol_lectura;
GRANT

practica1_db=# GRANT INSERT, UPDATE, DELETE ON TABLE pruebas_final TO
rol_escritura;
GRANT
```

A continuación se ejecutaron pruebas para validar la correcta configuración de los permisos:

1. **Prueba de lectura:** El usuario **consultor** y **editor** pueden realizar consultas SELECT sobre la tabla de **pruebas_final** exitosamente.
2. **Prueba de escritura (intento de violación de seguridad):** Al intentar realizar una inserción con el usuario **consultor**, el SGBD rechazó la operación lanzando el error **permission denied**, tal y como se observa a continuación, validando así la restricción de privilegios. Mientras que, cuando se intentó con **editor** la operación se pudo completar exitosamente.

```

root@core ~/practica1/postgres# docker exec -it postgres-practica1 psql -U
consultor -d practica1_db
psql (15.15 (Debian 15.15-1.pgdg13+1))
Type "help" for help.

practica1_db=> SELECT * FROM pruebas_final;
 id |      dato
----+-----
  1 | Dato visible 1
  2 | Dato visible 2
(2 rows)

practica1_db=> INSERT INTO pruebas_final (dato) VALUES ('Intento hacker');
ERROR:  permission denied for table pruebas_final

practica1_db=> \q

root@core ~/practica1/postgres# docker exec -it postgres-practica1 psql -U
editor -d practica1_db

practica1_db=> SELECT * FROM pruebas_final;
 id |      dato
----+-----
  1 | Dato visible 1
  2 | Dato visible 2
(2 rows)

practica1_db=> INSERT INTO pruebas_final (dato) VALUES ('Intento hacker');
INSERT 0 1

```

3.4. Estrategia de copia de seguridad

Se ha implementado un mecanismo de respaldo lógico utilizando la herramienta nativa **pg_dump**. El comando ejecutado desde el host de Docker permite extraer la estructura y datos de la base de datos **practica1_db** a un fichero .sql en el sistema local, garantizando la recuperación ante desastres.

```

root@core ~/practica1/postgres# docker exec -t postgres-practica1 pg_dump -U
alumno -d practica1_db > backup_postgre.sql

```

Para verificar que el fichero se generó correctamente y documentar su contenido, se inspeccionaron las primeras líneas del volcado:

```
root@core ~/practica1/postgres# head -n 15 backup_postgre.sql
--
-- PostgreSQL database dump
--

\restrict IeGpvOy3L1ZxaNDgFKLXBN4zMbQsMVyaB1VchG7lDNhKfGDBwqAa13C6tfft10k
-- Dumped from database version 15.15 (Debian 15.15-1.pgdg13+1)
-- Dumped by pg_dump version 15.15 (Debian 15.15-1.pgdg13+1)
SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
```

El encabezado del fichero confirma que se trata de un volcado generado por `pg_dump` e incluye, además, información relevante para la trazabilidad y la reproducibilidad del respaldo. En particular, se documentan la **versión del servidor** y la **versión de la herramienta** empleada, lo que permite evaluar compatibilidad en futuras restauraciones. Asimismo, se incorporan sentencias `SET` que establecen parámetros de sesión (por ejemplo, codificación, timeouts y `search_path`) para que la restauración se ejecute en un entorno controlado y lo más independiente posible de la configuración del servidor de destino.

3.5. Generación de datos y pruebas

Para evaluar la funcionalidad básica del SGBD, se ha diseñado un esquema conceptual común basado en la gestión de una biblioteca (lectores, libros y préstamos). Este diseño permite validar el soporte del motor relacional para el control de integridad referencial (claves primarias y foráneas).

Análisis de estrategias de generación de datos: Para poblar bases de datos relacionales estandarizadas como PostgreSQL, se ha analizado el uso de herramientas externas de generación de datos mock (datos ficticios).

- **Herramientas evaluadas:** Aplicaciones web como **Mockaroo** [8] o librerías de Python como **Faker** [9].
- **Ventaja:** Permiten generar de forma rápida miles de sentencias `INSERT` con datos semánticamente coherentes (nombres reales, correos, ISBNs válidos) exportadas directamente en un fichero `.sql` listo para su ejecución.
- **Implementación:** Se ha utilizado este enfoque simulado para generar el script de inicialización que el usuario **editor** ejecuta en la base de datos.

Código de pruebas:

```
-- 1. Definición del esquema relacional
CREATE TABLE lectores (
    dni VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(50),
    email VARCHAR(50)
);

CREATE TABLE libros (
    isbn VARCHAR(20) PRIMARY KEY,
    titulo VARCHAR(100),
    autor VARCHAR(50)
);

CREATE TABLE prestamos (
    id SERIAL PRIMARY KEY,
    dni_lector VARCHAR(15) REFERENCES lectores(dni),
    isbn_libro VARCHAR(20) REFERENCES libros(isbn),
    fecha_prestamo DATE
);

-- 2. Inserción de datos (Simulando la salida de un script de Mockaroo)
INSERT INTO lectores VALUES ('12345678A', 'Laura Gómez', 'laura@email.com');
INSERT INTO libros VALUES ('978-84-376-0494-7', 'Cien años de soledad', 'Gabriel
García Márquez');
INSERT INTO prestamos (dni_lector, isbn_libro, fecha_prestamo)
VALUES ('12345678A', '978-84-376-0494-7', '2026-02-20');

-- 3. Consulta de prueba (Verificación de relaciones mediante JOIN)
SELECT l.nombre AS Lector, b.titulo AS Libro, p.fecha_prestamo
FROM prestamos p
JOIN lectores l ON p.dni_lector = l.dni
JOIN libros b ON p.isbn_libro = b.isbn;
```

4. Oracle XE

4.1. Definición del servicio y control de versiones

Para el despliegue de Oracle se ha utilizado la imagen oficial de **Oracle Container Registry** correspondiente a **Oracle Database Express Edition**.

A diferencia de otros SGBD, Oracle requiere una inicialización interna compleja, por lo que es imprescindible utilizar volumen persistente y permitir un tiempo de arranque prolongado.

```
root@core ~# mkdir -p ~/practical1/oracle
root@core ~# cd ~/practical1/oracle
```

Dentro de este directorio se crea el fichero docker-compose.yml asociado a este SGDB concreto y cuyo contenido se muestra a continuación:

```

version: '3.8'
networks:
  bd2_network:
    driver: bridge
services:
  oracle:
    image: container-registry.oracle.com/database/express:latest
    container_name: oracle-xe
    networks:
      - bd2_network
    environment:
      - ORACLE_PWD=oracle123
    ports:
      - "1521:1521"
      - "5500:5500"
    volumes:
      - oracle_data:/opt/oracle/oradata

volumes:
  oracle_data:

```

Justificación de la configuración:

- Puerto 1521 → Listener Oracle (conexión SQL).
- Puerto 5500 → Oracle Enterprise Manager Express.
- Volumen oracle_data → Persistencia del diccionario de datos y esquemas.

El servicio se despliega del siguiente modo:

```

root@core ~/practical1/oracle# docker-compose up -d
WARN[0000] /root/practical1/oracle/docker-compose.yml: the attribute `version` is
obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 8/8
✓ Image container-registry.oracle.com/database/express:latest Pulled    1389.6s
✓ Network oracle_bd2_network Created      0.4s
✓ Volume oracle_oracle_data Created      0.0s
✓ Container oracle-xe Created          55.7s

```

4.2. Configuración de seguridad y gestión de roles

Una vez desplegado el motor de base de datos en el contenedor, es necesario disponer de herramientas que permitan la administración y conectividad. Aunque es posible administrar Oracle desde el interior del contenedor, se ha optado por instalar **Oracle Instant Client** en la máquina virtual, que funcione como host de los contenedores. Esta configuración es fundamental por dos motivos: permite el uso de la herramienta sqlplus directamente desde la terminal de la VM y facilita la validación de la conectividad por red antes de pasar a herramientas gráficas como DBeaver.

4.2.1. Descarga e instalación de componentes

Se han descargado los paquetes Basic y SQLPlus* de la versión 23.7 (compatible con el motor 21c). El proceso se realizó siguiendo estos comandos:

1. Preparación del directorio y descarga del paquete básico

```
mkdir -p $HOME/instantclient_23_7

cd ~/practical1/oracle

wget
https://download.oracle.com/otn_software/linux/instantclient/2370000/instantclient-ba
sic-linux.x64-23.7.0.25.01.zip

unzip instantclient-basic-linux.x64-23.7.0.25.01.zip -d $HOME/instantclient_23_7
```

2. Descarga e integración de SQL*Plus:

```
wget
https://download.oracle.com/otn_software/linux/instantclient/2370000/instantclient-sq
lplus-linux.x64-23.7.0.25.01.zip

unzip instantclient-sqlplus-linux.x64-23.7.0.25.01.zip -d $HOME/instantclient_23_7
```

4.2.2. Configuración de variables de entorno

Para que el sistema reconozca las librerías y ejecutables de Oracle, es indispensable configurar las variables de entorno. Se han añadido las siguientes líneas al fichero ~/.bashrc:

```
# Definición de rutas para Oracle Instant Client
export ORACLE_HOME=$HOME/instantclient_23_7/instantclient_23_7
export LD_LIBRARY_PATH=$ORACLE_HOME
export PATH=$PATH:$ORACLE_HOME
```

Tras la edición, se refresca la sesión para aplicar los cambios:

```
source ~/.bashrc
```

4.2.3. Validación de la herramienta cliente

Con esta configuración, la máquina virtual ya dispone de la capacidad de conectar con el contenedor de Oracle. La importancia de este paso radica en que proporciona una capa de abstracción: el administrador puede gestionar el SGBD como si estuviera instalado localmente en la VM, utilizando el comando sqlplus junto con la dirección IP del contenedor o localhost gracias al mapeo de puertos realizado en el paso 4.1.

Esta infraestructura cliente es la que permitirá, en el siguiente apartado, realizar la conexión inicial como **SYSDBA** para configurar la seguridad del sistema.

4.2.4. Inicialización, gestión de seguridad y roles

Tras la correcta instalación del cliente y el despliegue del contenedor, el siguiente paso consiste en la inicialización lógica de la base de datos. En Oracle 21c XE, esto implica diferenciar entre la base de datos raíz (CDB) y la base de datos enchufable de usuario (**XEPDB1**), donde se almacenarán los datos de la práctica.

Debido a que las imágenes oficiales de Oracle en Docker vienen con las cuentas administrativas bloqueadas por motivos de seguridad, el primer paso es establecer las credenciales. Para ello, se accede de forma interactiva a la terminal interna del contenedor, cuyo prompt nativo es `bash-4.2$`, y se ejecuta el script de configuración provisto por la imagen oficial:

```
root@core ~/practica1/oracle# docker exec -it oracle-xe bash
bash-4.2$ ls
setPassword.sh
bash-4.2$ ./setPassword.sh oracle123
The Oracle base remains unchanged with value /opt/oracle

SQL*Plus: Release 21.0.0.0.0 - Production on Thu Feb 19 08:59:04 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> exit
Disconnected from Oracle Database 21c Express Edition...
```

Una vez establecida la contraseña para el usuario **SYS**, se procede a realizar la conexión mediante la herramienta `sqlplus` instalada en el apartado 4.2. Es fundamental especificar la conexión como `sysdba` para tener privilegios totales de administración:

```
bash-4.2$ sqlplus sys/oracle123@//localhost:1521/XE as sysdba

SQL*Plus: Release 21.0.0.0.0 - Production on Thu Feb 19 09:14:11 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL>
```

Al conectar por defecto al contenedor raíz, se ejecuta el siguiente comando para trasladar la sesión a la base de datos de usuario donde se definirán los roles:

```
ALTER SESSION SET CONTAINER = XEPDB1;
```

Al igual que en el SGBD de PostgreSQL se siguió el mismo diseño para la creación de roles, asignación de privilegios a dichos roles y gestión de usuarios.

```
-- 1. Creación de roles
CREATE ROLE rol_lectura;
CREATE ROLE rol_escritura;

-- 2. Asignación de permisos a los roles
GRANT CREATE SESSION TO rol_lectura; -- Permite conectar (loguearse)

-- Permisos de escritura (Hereda de lectura y añade capacidad de creación)
GRANT rol_lectura TO rol_escritura;
GRANT RESOURCE TO rol_escritura; -- Rol del sistema que permite crear objetos
GRANT CREATE TABLE TO rol_escritura; -- Permiso explícito para tablas

-- 3. Creación y asignación de usuarios
-- Superusuario (Alumno)
CREATE USER alumno IDENTIFIED BY alumno123;
GRANT DBA TO alumno;

-- Editor (Escritura: hereda rol_escritura)
CREATE USER editor IDENTIFIED BY editor123;
GRANT rol_escritura TO editor;
ALTER USER editor QUOTA UNLIMITED ON USERS; -- Clave: permite usar espacio en disco

-- Consultor (Lectura: hereda rol_lectura)
CREATE USER consultor IDENTIFIED BY consultor123;
GRANT rol_lectura TO consultor;
```

A diferencia de PostgreSQL, Oracle Database aplica una política de seguridad mucho más estricta donde los roles recién creados carecen totalmente de permisos. Por ello, fue necesario otorgar explícitamente el privilegio de sistema `CREATE SESSION` al `rol_lectura` para permitir la simple conexión al motor. Para el `rol_escritura`, además de heredar los permisos de lectura, se le asignó el rol predefinido `RESOURCE`, el cual empaqueta los privilegios necesarios para que el motor permita la creación y gestión de estructuras lógicas como tablas y secuencias.

La diferencia más crítica respecto a otros SGBD relacionales se encuentra en la gestión de los usuarios y el almacenamiento físico. En Oracle existe una separación estricta entre el permiso lógico para crear una tabla y el permiso físico para escribir datos en ella. Por este motivo, al configurar el usuario **editor**, fue un paso obligatorio asignarle una cuota de disco explícita mediante la sentencia `ALTER USER editor QUOTA UNLIMITED ON USERS;`. Sin esta directiva, cualquier intento de realizar un `INSERT` habría sido bloqueado por el sistema por falta de espacio en el tablespace, garantizando así un control granular sobre el consumo de recursos.

4.3. Verificación de políticas de seguridad

Para comprobar la correcta configuración del modelo de roles y permisos, se ha seguido la misma batería de pruebas diseñada para el SGBD PostgreSQL (cuyo planteamiento metodológico ya se ha detallado en su respectivo apartado). Por tanto, en esta sección se muestran directamente los resultados de la ejecución de dichas pruebas adaptadas a Oracle.

La principal diferencia operativa en esta fase respecto a Postgres es que, gracias a la asignación de cuota de disco y del rol RESOURCE, el usuario **editor** tiene autonomía para crear la tabla productos directamente en su propio esquema. De este modo, el superusuario **alumno** (DBA) no necesita intervenir en la definición de la estructura de datos.

A continuación, se muestran las evidencias de ejecución. Primero, el usuario editor crea los datos y cede los permisos:

```
bash-4.2$ sqlplus editor/editor123@//localhost:1521/XEPDB1

SQL*Plus: Release 21.0.0.0.0 - Production on Thu Feb 19 09:36:27 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> CREATE TABLE productos (id NUMBER, nombre VARCHAR2(50));

Table created.

SQL> INSERT INTO productos VALUES (1, 'Monitor 4K');

1 row created.

SQL> COMMIT;

Commit complete.

SQL> GRANT SELECT ON productos TO rol_lectura;

Grant succeeded.

SQL> SELECT * FROM productos;

      ID NOMBRE
-----
      1 Monitor 4K
```

Posteriormente, se comprueba que el **consultor** tiene acceso de lectura pero es bloqueado al intentar escribir:

```

bash-4.2$ sqlplus consultor/consultor123@//localhost:1521/XEPDB1

SQL*Plus: Release 21.0.0.0.0 - Production on Thu Feb 19 09:39:06 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> SELECT * FROM editor.productos;

      ID NOMBRE
-----
      1 Monitor 4K

SQL> INSERT INTO editor.productos VALUES (2, 'Teclado RGB');
INSERT INTO editor.productos VALUES (2, 'Teclado RGB')
*
ERROR at line 1:
ORA-01031: insufficient privileges

```

A diferencia de PostgreSQL, donde los permisos pueden gestionarse de forma más general a nivel de esquema global, en Oracle predomina el concepto de "propietario de esquema". Aunque el usuario **consultor** posea el rol_lectura, el **editor** (como creador y propietario de los datos) ha tenido que autorizar explícitamente el acceso a su tabla mediante el comando GRANT SELECT, garantizando así un control de acceso mucho más granular y seguro por diseño. El error **ORA-01031** confirma que la barrera de seguridad de escritura funciona correctamente.

4.4. Estrategia de copia de seguridad

Para conseguir realizar un volcado de datos, se ha utilizado la herramienta oficial **Oracle Data Pump (expdp)**. Debido a la arquitectura de bases de datos enchufables de Oracle 21c XE y al aislamiento del sistema de archivos del contenedor Docker, el proceso requiere una preparación previa consistente en la creación de un directorio lógico.

A continuación, se detalla paso a paso el proceso seguido y el propósito de cada comando ejecutado:

1. Creación del directorio lógico y asignación de permisos

En Oracle, las herramientas de exportación no escriben directamente en rutas físicas del sistema operativo por motivos de seguridad, sino en directorios lógicos definidos internamente. Para configurarlo, es necesario acceder con el superusuario (SYSDBA).

- Mediante el comando `docker exec -it oracle-xe sqlplus sys/oracle123@//localhost:1521/XEPDB1 as sysdba` se abre una sesión interactiva con máximos privilegios conectada a la base de datos de usuario XEPDB1.

- La instrucción `CREATE OR REPLACE DIRECTORY DIR_BACKUP AS '/opt/oracle/oradata';` crea un puntero lógico llamado `DIR_BACKUP` que enlaza con la ruta física `/opt/oracle/oradata`. Esta ruta es clave, ya que es el volumen persistente montado en el `docker-compose.yml` de Oracle.
- Seguidamente, `GRANT READ, WRITE ON DIRECTORY DIR_BACKUP TO editor;` autoriza explícitamente al usuario **editor** a leer y escribir ficheros en dicho directorio, requisito indispensable para que pueda exportar sus propias tablas. Finalmente, se cierra la sesión con `exit;`.

```
root@core ~/practical1/oracle# docker exec -it oracle-xe sqlplus
sys/oracle123@//localhost:1521/XEPDB1 as sysdba

SQL*Plus: Release 21.0.0.0.0 - Production on Fri Feb 20 15:40:22 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> CREATE OR REPLACE DIRECTORY DIR_BACKUP AS '/opt/oracle/oradata';

Directory created.

SQL> GRANT READ, WRITE ON DIRECTORY DIR_BACKUP TO editor;

Grant succeeded.

SQL> exit;
Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 -
Production
Version 21.3.0.0.0
```

2. Exportación

Una vez preparado el entorno, se procede a invocar la utilidad `expdp` (Export Data Pump) desde el exterior del contenedor mediante `docker exec`. Los parámetros proporcionados configuran el comportamiento de la exportación:

- **editor/editor123@//localhost:1521/XEPDB1:** Credenciales y cadena de conexión que indican que la operación la realiza el propietario de los datos.
- **DIRECTORY=DIR_BACKUP:** Indica al motor de Oracle que utilice el directorio lógico creado en el paso anterior como destino de los ficheros.
- **DUMPFILE=backup_editor.dmp:** Define el nombre del archivo binario resultante que contendrá la estructura y los datos.

- **LOGFILE=backup_editor.log:** Genera un archivo de texto con el registro detallado de las operaciones, útil para verificar que no ha habido errores.
- **TABLES=productos:** Restringe el alcance de la exportación únicamente a la tabla indicada (volcado parcial de la estructura lógica y física).

```

root@core ~/practica1/oracle# docker exec -it oracle-xe expdp
editor/editor123@//localhost:1521/XEPDB1 DIRECTORY=DIR_BACKUP
DUMPFILE=backup_editor.dmp LOGFILE=backup_editor.log TABLES=productos

Export: Release 21.0.0.0.0 - Production on Fri Feb 20 15:41:39 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Connected to: Oracle Database 21c Express Edition Release 21.0.0.0.0 -
Production
Starting "EDITOR"."SYS_EXPORT_TABLE_01":
editor/*****@//localhost:1521/XEPDB1 DIRECTORY=DIR_BACKUP
DUMPFILE=backup_editor.dmp LOGFILE=backup_editor.log TABLES=productos
Processing object type TABLE_EXPORT/TABLE/TABLE_DATA
Processing object type TABLE_EXPORT/TABLE/STATISTICS/TABLE_STATISTICS
Processing object type TABLE_EXPORT/TABLE/STATISTICS/MARKER
Processing object type TABLE_EXPORT/TABLE/TABLE
Processing object type TABLE_EXPORT/TABLE/GRANT/OWNER_GRANT/OBJECT_GRANT
. . exported "EDITOR"."PRODUCTOS"                    5.492 KB          1 rows
Master table "EDITOR"."SYS_EXPORT_TABLE_01" successfully loaded/unloaded
*****
Dump file set for EDITOR.SYS_EXPORT_TABLE_01 is:
/opt/oracle/oradata/backup_editor.dmp
Job "EDITOR"."SYS_EXPORT_TABLE_01" successfully completed at Fri Feb 20 15:42:20
2026 elapsed 0 00:00:36

```

Además, si se presta atención al resultado de esta ejecución se puede observar que en el proceso de exportación de la tabla de *productos* de la cual es propietario **editor** se ha exportado exitosamente una línea, lo cual coincide con el tamaño total de dicha tabla, que tiene como contenido: 1 Monitor 4K.

3. Extracción del archivo de copia de seguridad al sistema anfitrión

Aunque el archivo backup_editor.dmp se ha guardado en un volumen persistente, es más apropiado extraer la copia de seguridad del entorno del SGBD para garantizar su disponibilidad. Para ello, se emplea el comando siguiente, que consiste en copiar el fichero binario generado desde el interior del contenedor hacia el directorio actual de la máquina virtual.

```

root@core ~/practica1/oracle# docker cp
oracle-xe:/opt/oracle/oradata/backup_editor.dmp ./backup_editor.dmp
Successfully copied 182kB to /root/practica1/oracle/backup_editor.dmp

```

4.5. Generación de datos y pruebas

En Oracle se ha desplegado el mismo esquema relacional de biblioteca, pero aprovechando las capacidades internas avanzadas del motor para la generación masiva de la información, en lugar de depender de ficheros externos.

Análisis de estrategias de generación de datos: Frente a la dependencia de herramientas externas evaluada en Postgres, en Oracle se ha optado por una estrategia de **Generación nativa en el servidor mediante PL/SQL**.

- **Ventaja:** Al delegar la lógica de generación al propio motor relacional mediante bucles y paquetes internos como DBMS_RANDOM, se elimina por completo la latencia de red. Es la estrategia más eficiente para realizar pruebas de estrés y evaluar la volumetría masiva.
- **Implementación:** Tras insertar manualmente el catálogo base de libros y lectores, se utiliza un bloque PL/SQL para generar transacciones de préstamos aleatorias masivas en cuestión de milisegundos.

Código de pruebas:

```
-- 1. Definición del esquema (Adaptado a sintaxis Oracle)
CREATE TABLE lectores (dni VARCHAR2(15) PRIMARY KEY, nombre VARCHAR2(50));
CREATE TABLE libros (isbn VARCHAR2(20) PRIMARY KEY, titulo VARCHAR2(100));
CREATE TABLE prestamos (
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    dni_lector VARCHAR2(15) REFERENCES lectores(dni),
    isbn_libro VARCHAR2(20) REFERENCES libros(isbn),
    fecha_prestamo DATE
);

-- 2. Inserción de catálogo base
INSERT INTO lectores VALUES ('12345678A', 'Laura Gómez');
INSERT INTO libros VALUES ('978-84', 'Cien años de soledad');
INSERT INTO libros VALUES ('978-99', 'El Quijote');

-- 3. Estrategia PL/SQL: Generación masiva de 500 préstamos aleatorios en el
servidor
BEGIN
    FOR i IN 1..500 LOOP
        INSERT INTO prestamos (dni_lector, isbn_libro, fecha_prestamo)
        VALUES (
            '12345678A',
            CASE ROUND(DBMS_RANDOM.VALUE(1,2)) WHEN 1 THEN '978-84' ELSE
'978-99' END,
            SYSDATE - ROUND(DBMS_RANDOM.VALUE(1, 30))
        );
    END LOOP;
    COMMIT;
END;
/
```

```
-- 4. Consulta analítica de comprobación (Libros más prestados)
SELECT isbn_libro, COUNT(*) as Total_Prestamos
FROM prestamos
GROUP BY isbn_libro
ORDER BY Total_Prestamos DESC;
```

5. Apache Cassandra

5.1. Definición del servicio y control de versiones

Para el despliegue del sistema NoSQL Apache Cassandra, se ha utilizado la imagen oficial en su última versión. Se ha definido un entorno aislado mediante una red bridge y se ha habilitado la persistencia de datos.

```
root@core ~# mkdir -p ~/practical/cassandra
root@core ~# cd ~/practical/cassandra
```

Tras esto, se crea el fichero de configuración exponiendo el puerto estándar **9042**, utilizado por el protocolo nativo de Cassandra (CQL).

```
version: '3.8'

networks:
  bd2_network:
    driver: bridge

services:
  cassandra:
    image: cassandra:latest
    container_name: cassandra-db
    networks:
      - bd2_network
    ports:
      - "9042:9042" # Puerto estándar de Cassandra (CQL)
    environment:
      - CASSANDRA_CLUSTER_NAME=Practica1_Cluster
    volumes:
      - cassandra_data:/var/lib/cassandra

volumes:
  cassandra_data:
```

Justificación de la configuración:

- **environment:** Se ha establecido la variable `CASSANDRA_CLUSTER_NAME` para definir el nombre lógico del clúster de nodos (**Practica1_Cluster**), lo cual es una buena práctica para sobrescribir el valor por defecto en esta base de datos distribuida.

- **volumes:** Se ha definido un volumen nombrado (**cassandra_data**) montado en la ruta `/var/lib/cassandra`. Esto asegura que la información almacenada (como el Keyspace, las tablas y los usuarios) sobreviva al reinicio o eliminación del contenedor.

El despliegue se realiza mediante:

```
root@core ~/practica1/cassandra# docker-compose up -d
```

5.2. Configuración de seguridad y roles

Una particularidad de **Apache Cassandra** es que, por defecto, se instala con una política de seguridad completamente abierta. El sistema permite que cualquier usuario acceda como anónimo (sin contraseña) mediante las clases `AllowAllAuthenticator` y `AllowAllAuthorizer`. Sin embargo, el usuario anónimo carece de privilegios para gestionar esquemas o crear otros roles, devolviendo un error de tipo **Unauthorized** ante comandos de administración.

Para cumplir con los requisitos de la práctica, fue imprescindible modificar la configuración interna del motor (fichero `cassandra.yaml`) para forzar la autenticación por contraseña y habilitar el gestor de permisos real. Esto se logró ejecutando los siguientes comandos desde la máquina host:

```
# 1. Activar la autenticación por contraseña
root@core ~/practica1/cassandra# docker exec -u root -it cassandra-db sed -i
's/authenticator: AllowAllAuthenticator/authenticator: PasswordAuthenticator/'
/etc/cassandra/cassandra.yaml

# 2. Activar la autorización (gestión de permisos)
root@core ~/practica1/cassandra# docker exec -u root -it cassandra-db sed -i
's/authorizer: AllowAllAuthorizer/authorizer: CassandraAuthorizer/'
/etc/cassandra/cassandra.yaml

# 3. Reiniciar el contenedor para aplicar los cambios
root@core ~/practica1/cassandra# docker restart cassandra-db
```

Tras activar la seguridad, el acceso anónimo queda restringido. Para inicializar el sistema, se accede utilizando el superusuario administrador que Cassandra genera por defecto (`cassandra / cassandra`).

```
root@core ~/practica1/cassandra# docker exec -it cassandra-db cqlsh -u cassandra
-p cassandra
```

```
Warning: Using a password on the command line interface can be insecure.
Recommendation: use the credentials file to securely provide the password.
```

```
Connected to Practica1_Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cassandra@cqlsh>
```

Una vez dentro, se procede a crear el espacio de nombres (Keyspace) y a definir el modelo de Control de Acceso (RBAC). En Cassandra, los roles y los usuarios son internamente la misma entidad, pero la sintaxis permite diferenciarlos de forma lógica:

```
-- 1. Creación del espacio de nombres (Keyspace)
CREATE KEYSPACE practica1_keyspace
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

-- 2. Creación de Roles Lógicos
CREATE ROLE rol_lectura;
CREATE ROLE rol_escritura;

-- 3. Creación de Usuarios
CREATE USER alumno WITH PASSWORD 'aLumno123' SUPERUSER;
CREATE USER editor WITH PASSWORD 'editor123';
CREATE USER consultor WITH PASSWORD 'consultor123';

-- 4. Asignación de Permisos a Los ROLES
GRANT ALL PERMISSIONS ON KEYSPACE practica1_keyspace TO rol_escritura;
GRANT SELECT ON KEYSPACE practica1_keyspace TO rol_lectura;

-- 5. Asignación de Roles a Los USUARIOS
GRANT rol_escritura TO editor;
GRANT rol_lectura TO consultor;
```

5.3. Verificación de políticas de seguridad

En esta sección se muestran directamente los resultados de la ejecución de las pruebas adaptadas a la sintaxis CQL de Apache Cassandra, ya que las pruebas realizadas son siempre las mismas y ya se han explicado en SGBDs anteriores.

La principal diferencia operativa en esta fase respecto a los sistemas relacionales clásicos (como Oracle o Postgres) es que la autorización en Cassandra se ha configurado a nivel global de **Keyspace** (espacio de nombres). Dado que el usuario **editor** posee todos los permisos (ALL PERMISSIONS) sobre el Keyspace, tiene autonomía para crear la tabla productos e insertar datos. A su vez, como el **consultor** ya recibió el permiso SELECT genérico sobre todo el Keyspace, no es necesario que el editor realice un GRANT explícito sobre la tabla recién creada; el **consultor** hereda la capacidad de lectura automáticamente.

A continuación, se muestran las evidencias de ejecución. Primero, el usuario **editor** crea la tabla de datos y realiza una inserción:

```
root@core ~/practica1/cassandra# docker exec -it cassandra-db cqlsh -u editor -p editor123
```

```
Warning: Using a password on the command line interface can be insecure.
Recommendation: use the credentials file to securely provide the password.
```

```
Connected to Practica1_Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
```

```

Use HELP for help.
editor@cqlsh> USE practica1_keyspace;
editor@cqlsh:practica1_keyspace> CREATE TABLE productos (
    ...     id int PRIMARY KEY,
    ...     nombre text,
    ...     precio float
    ... );
editor@cqlsh:practica1_keyspace> INSERT INTO productos (id, nombre, precio)
VALUES (1, 'Ratón Gamer', 45.99);
editor@cqlsh:practica1_keyspace> SELECT * FROM productos;

id | nombre      | precio
----+-----+-----
  1 | Ratón Gamer | 45.99

(1 rows)

```

Posteriormente, se comprueba que el consultor tiene acceso de lectura efectivo, pero es bloqueado de inmediato por el sistema al intentar escribir:

```

root@core ~/practica1/cassandra# docker exec -it cassandra-db cqlsh -u consultor
-p consultor123

Warning: Using a password on the command line interface can be insecure.
Recommendation: use the credentials file to securely provide the password.

Connected to Practica1_Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
consultor@cqlsh> USE practica1_keyspace;
consultor@cqlsh:practica1_keyspace> SELECT * FROM productos;

id | nombre      | precio
----+-----+-----
  1 | Ratón Gamer | 45.99

(1 rows)

consultor@cqlsh:practica1_keyspace> INSERT INTO productos (id, nombre, precio)
VALUES (2, 'Alfombrilla', 15.00);
Unauthorized: Error from server: code=2100 [Unauthorized] message="User
consultor has no MODIFY permission on <table practica1_keyspace.productos> or
any of its parents"

```

A diferencia de PostgreSQL u Oracle, donde los permisos de inserción, actualización y borrado se pueden otorgar de forma fragmentada, el modelo de seguridad de Cassandra (gestionado internamente por el módulo CassandraAuthorizer) agrupa todas las operaciones de escritura en disco bajo un único permiso global denominado MODIFY. Al intentar ejecutar la instrucción INSERT, el motor verifica el árbol de privilegios y detecta que el usuario consultor carece de esta directiva. El error devuelto (**code=2100 [Unauthorized]**)

confirma que la barrera de seguridad funciona a la perfección, protegiendo la integridad estructural y de datos del sistema NoSQL frente a usuarios de solo lectura.

5.4. Estrategia de copia de seguridad

A diferencia de los SGBD relacionales tradicionales que empaquetan la estructura y los datos en un único archivo binario propietario, en Apache Cassandra la estrategia estándar de volcado lógico (logical dump) implica separar la definición estructural (DDL) de los datos físicos.

Para garantizar una copia de seguridad completa, portable y fácilmente verificable, se han ejecutado comandos a través de la interfaz cqlsh, redirigiendo la salida estándar hacia ficheros locales en la máquina anfitriona.

A continuación, se detalla el proceso:

1. Volcado de la estructura lógica

En primer lugar, hay que extraer la definición del Keyspace y de todas las tablas que contiene. Para ello, se utiliza:

```
root@core ~/practica1/cassandra# docker exec -it cassandra-db cqlsh -u editor -p editor123 -e "DESCRIBE KEYSPACE practica1_keyspace;" > backup_esquema.cql
```

Este comando permite invocar la consola de Cassandra de forma no interactiva mediante el parámetro -e. Además, con DESCRIBE KEYSPACE practica1_keyspace; le indica al motor que devuelva todo el código CQL necesario para recrear el esquema desde cero y el operador > dirige esta salida al fichero de texto plano backup_esquema.cql.

2. Volcado físico de los datos

Una vez salvaguardada la estructura, se procede a extraer los registros de la tabla productos.

```
root@core ~/practica1/cassandra# docker exec -it cassandra-db cqlsh -u editor -p editor123 -e "COPY practica1_keyspace.productos TO STDOUT WITH HEADER = TRUE;" > backup_datos.csv
```

El comando COPY está altamente optimizado en Cassandra para la exportación masiva. Al enviarlo a la salida estándar en lugar de a un archivo interno del contenedor, se evitan problemas de permisos y se captura el flujo directamente en el host anfitrión, guardándolo en el archivo backup_datos.csv.

Con el objetivo de asegurar la correcta copia de los datos de la tabla productos dentro del fichero backup_datos.csv se hizo uso de cat.

```
root@core ~/practica1/cassandra# cat backup_datos.csv
```

```
Warning: Using a password on the command line interface can be insecure.  
Recommendation: use the credentials file to securely provide the password.
```

```
id,nombre,precio
1,Ratón Gamer,45.99
```

5.5. Generación de datos y pruebas

El modelado en Cassandra supone un cambio de paradigma total respecto a PostgreSQL y Oracle. En Cassandra no existen las relaciones (JOIN ni FOREIGN KEY), ya que el diseño se orienta a las consultas. Esto obliga a desnormalizar los datos, uniendo la información del lector y del libro en una única tabla.

Análisis de estrategias de generación de datos: Dado que Cassandra no soporta lenguaje procedimental nativo para generar datos mediante bucles como Oracle, se analizaron dos alternativas:

1. Scripts asíncronos en Python utilizando el driver oficial de Cassandra.
2. **Generación de ficheros CSV (estrategia seleccionada):** El motor de Cassandra está altamente optimizado para la introducción masiva de datos en formato tabular. La mejor estrategia consiste en generar un fichero CSV externamente (por ejemplo, con Mockaroo) e importarlo mediante el comando nativo COPY.

Código de pruebas:

```
-- 1. Definición del esquema (Desnormalizado: Orientado a buscar el historial de un lector)
-- Se agrupa toda la información en una sola tabla sin Foreign Keys
CREATE TABLE historial_prestamos (
  dni_lector text,
  fecha_prestamo date,
  nombre_lector text,
  titulo_libro text,
  PRIMARY KEY (dni_lector, fecha_prestamo)
);

-- 2. Estrategia de Ingesta Masiva: Importación directa desde un CSV generado
-- (Este comando se ejecuta desde la herramienta interactiva cqlsh)
COPY historial_prestamos (dni_lector, fecha_prestamo, nombre_lector,
titulo_libro)
FROM 'prestamos_generados.csv' WITH HEADER = TRUE;

-- 3. Inserción manual de prueba (DML)
INSERT INTO historial_prestamos (dni_lector, fecha_prestamo, nombre_lector,
titulo_libro)
VALUES ('12345678A', '2026-02-20', 'Laura Gómez', 'Cien años de soledad');

-- 4. Consulta de prueba (La Primary Key obliga a consultar filtrando por la Partition Key)
```

```
SELECT fecha_prestamo, titulo_libro
FROM historial_prestamos
WHERE dni_lector = '12345678A';
```

6. Apache HBase

6.1. Definición del servicio y control de versiones

Para integrar Apache HBase en el entorno de la práctica de forma reproducible y coherente con el resto de SGBD desplegados, se optó por gestionarlo mediante Docker Compose. Con el fin de mantener una estructura ordenada y, especialmente, garantizar la persistencia de los datos fuera del ciclo de vida del contenedor, se preparó un directorio de trabajo específico (~/bd2/hbase) y un directorio local ./data destinado a actuar como volumen persistente.

```
root@core ~# mkdir -p ~/bd2/hbase
root@core ~# cd ~/bd2/hbase
root@core ~/bd2/hbase# mkdir -p data
```

A continuación, se definió el servicio en el fichero docker-compose.yml empleando la imagen dajobe/hbase, que proporciona un entorno funcional de HBase en modo standalone. La definición incluye una política de reinicio (unless-stopped) para facilitar la recuperación automática del servicio tras reinicios del sistema, la publicación del puerto 16010 para acceder a la interfaz web del HBase Master (útil como mecanismo de verificación y monitorización), y el montaje del volumen ./data sobre /opt/hbase-data para asegurar que la información se conserve aunque el contenedor se detenga o se recree.

```
services:
  hbase:
    image: dajobe/hbase
    container_name: hbase-db
    restart: unless-stopped
    ports:
      - "16010:16010" # Web UI del Master
    volumes:
      - ./data:/opt/hbase-data
```

Una vez definida la configuración, se desplegó el servicio y se comprobó su creación correcta dentro del entorno de Docker, lo que permite confirmar que tanto la imagen como la red del servicio han sido aprovisionadas sin errores.

```
root@core ~/bd2/hbase# docker compose up -d
[+] up 13/13
✓ Image dajobe/hbase      Pulled
✓ Network hbase_default   Created
✓ Container hbase-db      Created
```

Tras el despliegue, se verificó que el contenedor se encontraba en ejecución y que el puerto 16010 estaba efectivamente expuesto, evidenciando que el servicio arrancó y quedó accesible desde el host.

```
root@core ~/bd2/hbase# docker ps | grep hbase-db
5f3bce4bebda  dajobe/hbase  "/opt/hbase-server"  Up 40 seconds
0.0.0.0:16010->16010/tcp  hbase-db
```

Finalmente, para documentar la versión real del sistema ejecutado y validar el arranque correcto de los componentes internos, se revisaron las trazas de inicialización del contenedor. Esta verificación resulta útil en despliegues basados en imágenes Docker, ya que permite obtener evidencia directa de la versión y del modo de operación efectivo del entorno.

```
root@core ~/bd2/hbase# docker logs -f hbase-db
INFO  master.HMaster: STARTING service HMaster
INFO  util.VersionInfo: HBase 2.1.2
INFO  server.ZooKeeperServer: zookeeper.version=3.4.10
INFO  http.HttpServer: Jetty bound to port 16010
INFO  master.HMaster: hbase.rootdir=file:/data/hbase,
hbase.cluster.distributed=false
```

En la salida se identifica la versión de Apache HBase (2.1.2) y de ZooKeeper (3.4.10), la disponibilidad del servidor web en el puerto 16010 y, además, parámetros que describen la arquitectura efectiva del despliegue. En particular, el sistema se ejecuta en modo standalone (`hbase.cluster.distributed=false`) y utiliza almacenamiento sobre filesystem local (`hbase.rootdir=file:/data/hbase`), lo que implica que los componentes operan en un único nodo sin depender de un clúster Hadoop distribuido ni de HDFS. En consecuencia, el entorno puede interpretarse como un despliegue local con separación lógica de roles, donde el HMaster coordina el sistema, el RegionServer sirve y almacena los datos y ZooKeeper proporciona coordinación interna. Aunque en entornos productivos HBase suele operar sobre HDFS para habilitar escalabilidad horizontal y tolerancia a fallos, el modo standalone simplifica la configuración y resulta adecuado para prácticas, ya que permite trabajar con el modelo de datos y las herramientas administrativas sin la complejidad de un ecosistema Hadoop completo. Esta arquitectura difiere de SGBD relacionales como PostgreSQL o IBM Db2, donde el motor se presenta como un servicio centralizado, mientras que en HBase se mantiene la separación conceptual entre coordinación y servicio de datos, coherente con su orientación a sistemas distribuidos.

6.2. Creación de estructura de datos

Una vez verificado el funcionamiento del servicio, se definió una estructura de datos mínima en Apache HBase con el objetivo de validar el modelo de almacenamiento y disponer de una base funcional sobre la que realizar inserciones y consultas. A diferencia de los SGBD relacionales, HBase no trabaja con un esquema rígido de columnas predefinidas, sino que organiza la información en namespaces, tablas, familias de columnas y claves de fila. En este modelo, las familias de columnas se declaran al crear la tabla, pero las columnas individuales se materializan de forma dinámica al insertar datos, lo que refleja el carácter flexible del sistema dentro de cada familia.

Para operar con HBase en este despliegue se accedió al contenedor y se abrió la consola interactiva proporcionada por el propio sistema, que permite ejecutar las operaciones básicas de definición y manipulación de datos directamente contra el motor.

```
root@core ~/bd2/hbase# docker exec -it hbase-db bash
root@5f3bce4bebda /# hbase shell
hbase(main):001:0>
```

Con el fin de mantener los objetos organizados y separar el trabajo de pruebas del espacio por defecto, se creó un namespace denominado espacioHBase, que cumple un papel similar al de una base de datos lógica en sistemas relacionales. A continuación, se listaron los namespaces disponibles para comprobar que la operación se aplicó correctamente.

```
hbase(main):001:0> create_namespace 'espacioHBase'
Took 0.5573 seconds
hbase(main):002:0> list_namespace
NAMESPACE
default
espacioHBase
hbase
3 row(s)
Took 0.1207 seconds
```

Dentro del namespace creado se definió una tabla llamada pruebas_final junto con una familia de columnas info. En HBase la tabla queda identificada por la combinación namespace:tabla, y la declaración de la familia es el elemento “estable” del esquema. Las columnas concretas no se declaran en este punto, ya que se incorporarán implícitamente al insertar valores con su correspondiente calificador.

```
hbase(main):003:0> create 'espacioHBase:pruebas_final', 'info'
Created table espacioHBase:pruebas_final
Took 0.8445 seconds
=> Hbase::Table - espacioHBase:pruebas_final
hbase(main):004:0> list
TABLE
espacioHBase:pruebas_final
1 row(s)
Took 0.0427 seconds
=> ["espacioHBase:pruebas_final"]
```

A continuación se realizaron inserciones de prueba mediante put, que permite almacenar un valor en una celda identificada por la tabla, una row key y una columna expresada como familia:calificador. En este caso, las claves de fila 1 y 2 actúan como identificadores principales de cada registro y determinan cómo se direccionan y recuperan los datos. La columna utilizada fue info:dato, donde info corresponde a la familia declarada y dato al calificador dinámico creado en el momento de la inserción.

```
hbase(main):005:0> put 'espacioHBase:pruebas_final', '1', 'info:dato', 'Dato'
```

```
visible 1'
Took 0.2074 seconds

hbase(main):006:0> put 'espacioHBase:pruebas_final', '2', 'info:dato', 'Dato
visible 2'
Took 0.0108 seconds
```

Para verificar que las inserciones se habían aplicado correctamente, se utilizaron dos mecanismos de consulta complementarios. Por un lado, get recupera una fila concreta a partir de su clave, lo que permite validar el acceso directo por row key. Por otro, scan recorre la tabla completa y resulta útil para inspecciones generales. En ambos casos, la salida muestra el valor almacenado junto con la marca temporal asociada a la celda, característica habitual en HBase al tratarse de un sistema orientado a versiones temporales por celda.

```
hbase(main):007:0> get 'espacioHBase:pruebas_final', '1'
COLUMN          CELL
info:dato        timestamp=1771470493019, value=Dato visible 1
1 row(s)
Took 0.0516 seconds

hbase(main):008:0> scan 'espacioHBase:pruebas_final'
ROW      COLUMN+CELL
1        column=info:dato, timestamp=1771470493019, value=Dato visible 1
2        column=info:dato, timestamp=1771470500059, value=Dato visible 2
2 row(s)
Took 0.0495 seconds
```

Con estas operaciones se valida el flujo básico de trabajo en HBase en modo standalone, comprobando la creación de namespaces y tablas, el uso de familias de columnas y row keys como eje del acceso, y la inserción y consulta de datos sin necesidad de definir previamente un conjunto fijo de columnas, lo que ilustra la diferencia conceptual respecto a un modelo relacional clásico.

6.3. Seguridad en Apache HBase

Tras la creación de estructuras de datos y la verificación funcional del sistema, se analizó la viabilidad de implementar un modelo de control de acceso comparable al utilizado en SGBD relacionales, basado en usuarios con permisos diferenciados. En Apache HBase, la autorización granular se apoya en mecanismos de ACL (Access Control Lists) gestionados mediante el coprocesor AccessController y operaciones administrativas como grant. Por ello, como primera comprobación, se intentó aplicar permisos desde la consola interactiva de HBase. El intento falló y el propio sistema devolvió un mensaje explícito indicando que las funcionalidades de seguridad no estaban disponibles en el despliegue actual.

```
hbase(main):009:0> grant 'admin', 'RWXCA'
ERROR: DISABLED: Security features are not available
For usage try 'help "grant"'
```

Este resultado confirma que, en el entorno desplegado, las capacidades de autorización no están activas. A diferencia de PostgreSQL o IBM Db2, donde el modelo de roles y privilegios forma parte del núcleo del motor y está operativo por defecto, en HBase la autorización avanzada depende de la activación explícita de componentes y, en escenarios habituales, de una infraestructura de seguridad externa.

Con el objetivo de determinar si era posible habilitar dichas capacidades en el entorno de prácticas, se intentó activar manualmente el subsistema de autorización modificando la configuración del sistema. En este proceso se detectó primero un error metodológico. Se trató de ejecutar un comando del sistema (docker exec) desde la propia HBase Shell, lo que provocó un error de sintaxis al tratarse de una consola que únicamente interpreta comandos del entorno HBase/Ruby.

```
hbase(main):013:0> docker exec -it hbase-db bash
SyntaxError: (hbase):13: syntax error, unexpected tIDENTIFIER
```

Una vez corregido este punto, se salió de la shell y se accedió al contenedor desde la terminal del sistema, que es el contexto correcto para realizar tareas de administración del entorno Docker.

```
root@core ~/bd2/hbase# docker exec -it hbase-db bash
```

Dentro del contenedor, se localizó el directorio de configuración y se revisó el fichero principal hbase-site.xml. Antes de cualquier modificación se generó una copia de seguridad para garantizar reversibilidad y minimizar riesgos sobre la estabilidad del servicio.

```
root@5f3bce4bebda:/# ls /opt/hbase/conf
hbase-site.xml  hbase-site.xml.bak  hbase-policy.xml  ...

root@5f3bce4bebda:/# cat /opt/hbase/conf/hbase-site.xml

root@5f3bce4bebda:/# cp /opt/hbase/conf/hbase-site.xml
/opt/hbase/conf/hbase-site.xml.bak
```

A continuación, se reescribió la configuración añadiendo propiedades orientadas a activar autenticación simple y autorización, además de declarar el coprocesor AccessController a nivel de master y region server.

```
cat > /opt/hbase/conf/hbase-site.xml << 'EOF'
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>hbase.zookeeper.quorum</name>
    <value>5f3bce4bebda</value>
  </property>
  <property>
    <name>hbase.rootdir</name>
    <value>file:///data/hbase</value>
```

```
</property>
<property>
  <name>hbase.master.info.bindAddress</name>
  <value>5f3bce4bebdba</value>
</property>
<property>
  <name>hbase.regionserver.info.bindAddress</name>
  <value>5f3bce4bebdba</value>
</property>

<!-- Activación de seguridad -->
<property>
  <name>hbase.security.authentication</name>
  <value>simple</value>
</property>
<property>
  <name>hbase.security.authorization</name>
  <value>true</value>
</property>
<property>
  <name>hbase.coprocessor.master.classes</name>
  <value>org.apache.hadoop.hbase.security.access.AccessController</value>
</property>
<property>
  <name>hbase.coprocessor.region.classes</name>
  <value>org.apache.hadoop.hbase.security.access.AccessController</value>
</property>
</configuration>
EOF
```

Tras aplicar los cambios, se reinició el contenedor para forzar la recarga de la configuración.

```
root@core ~/bd2/hbase# docker restart hbase-db
```

Sin embargo, se constató que en HBase la seguridad no se habilita de forma efectiva únicamente mediante propiedades en hbase-site.xml. La documentación oficial describe que un despliegue con control de acceso completo suele requerir prerequisites arquitectónicos adicionales, en particular Kerberos (KDC) para autenticación y un entorno Hadoop asociado securizado, incluyendo HDFS y ZooKeeper [1]. En la práctica, esto implica disponer de un KDC operativo, principals y keytabs, y una configuración segura de los servicios dependientes, como ZooKeeper con autenticación y ACL (por ejemplo, SASL) [2], además del propio marco de seguridad descrito en la documentación histórica del proyecto [3]. En el entorno de prácticas utilizado, desplegado en modo standalone sobre Docker y con almacenamiento en filesystem local, no se dispone de dicha infraestructura. No existe servidor Kerberos ni material de autenticación (principals/keytabs), no se utiliza HDFS distribuido y ZooKeeper no está configurado con mecanismos de autenticación segura. Esta limitación tampoco se resolvería simplemente pasando a un despliegue pseudo-distribuido en una única máquina, ya que los requisitos de Kerberos y la securización de HDFS/ZooKeeper seguirían siendo necesarios para un modelo de autorización completo.

Dado que la activación del modelo de autorización no resultaba viable en este contexto y con el fin de mantener un despliegue estable, se restauró la configuración original utilizando la copia de seguridad del fichero y se reinició el contenedor para garantizar el retorno al estado inicial.

```
root@5f3bce4bebda:/# mv /opt/hbase/conf/hbase-site.xml.bak
/opt/hbase/conf/hbase-site.xml
root@core ~/bd2/hbase# docker restart hbase-db
```

En conclusión, en el despliegue standalone de HBase utilizado en la práctica no resulta razonable implementar un modelo de usuarios con privilegios diferenciados equivalente al aplicado en PostgreSQL e IBM Db2. La imposibilidad de activar ACL no se debe a un error puntual, sino a la ausencia de la infraestructura de autenticación y securización que normalmente acompaña a un entorno HBase/Hadoop completo. La documentación del intento, la modificación controlada con copia de seguridad y la posterior restauración reflejan un procedimiento técnico de análisis, prueba y decisión fundamentada.

6.4. Estrategia de copia de seguridad: Snapshots en HBase

Una vez validado el funcionamiento del sistema, se analizó el mecanismo de copia de seguridad disponible en Apache HBase desde un enfoque comparable al aplicado en los SGBD relacionales trabajados en la práctica. A diferencia de PostgreSQL, donde es habitual generar volcados lógicos en formato SQL mediante `pg_dump`, o de IBM Db2, que proporciona copias físicas con `BACKUP DATABASE`, HBase no produce respaldos en forma de script SQL debido a que su modelo de almacenamiento no es relacional. En su lugar, incorpora un mecanismo nativo denominado `snapshot`, orientado a capturar el estado consistente de una tabla en un instante determinado.

En HBase, un `snapshot` funciona como un punto de recuperación a nivel de tabla sin necesidad de duplicar íntegramente la información. En términos operativos, no se genera un “fichero exportable” equivalente a un `.sql`, sino una estructura interna asociada al almacenamiento del sistema. El `snapshot` se apoya en los ficheros internos (HFiles) y tiende a reutilizar datos existentes mediante referencias, lo que reduce el coste temporal y de espacio de la operación. La recuperación del estado capturado se realiza con comandos nativos del propio sistema, como `restore_snapshot`, por lo que el `snapshot` está diseñado principalmente para restauraciones dentro de entornos HBase compatibles y no como un artefacto textual portable.

Para validar este mecanismo en el entorno de prácticas, se generó un `snapshot` de la tabla creada previamente `espacioHBase:pruebas_final` y se verificó su existencia desde la shell de HBase.

```
hbase(main):004:0> snapshot 'espacioHBase:pruebas_final', 'snap_pruebas_final'
Took 0.6736 seconds

hbase(main):005:0> list_snapshots
SNAPSHOT                TABLE + CREATION TIME
snap_pruebas_final      espacioHBase:pruebas_final (2026-02-19 05:23:30 +0000)
1 row(s)
```

```
Took 0.0729 seconds  
=> ["snap_pruebas_final"]
```

La salida confirma que el snapshot `snap_pruebas_final` quedó asociado a la tabla indicada e incluye la marca temporal de creación. En caso de ser necesario recuperar el estado capturado, la restauración se realizaría mediante el comando `restore_snapshot`, que reconstruye el estado de la tabla a partir de la instantánea creada.

Desde un punto de vista comparativo, esta estrategia difiere de forma significativa de las empleadas en sistemas relacionales. En PostgreSQL y Db2 es habitual obtener copias lógicas o físicas como artefactos independientes que pueden transferirse con relativa facilidad entre entornos. En HBase, en cambio, el snapshot está ligado al almacenamiento interno del propio sistema y se concibe como un mecanismo de recuperación dentro del propio ecosistema HBase, sin generar un script reutilizable con sentencias SQL.

Por último, en el despliegue utilizado en la práctica, basado en modo standalone con almacenamiento local y persistencia mediante volumen Docker, los snapshots quedan almacenados bajo el directorio raíz definido por `hbase.rootdir` y se conservan mientras se mantenga el directorio persistente asociado en el host. Esto implica que, aunque el contenedor se detenga o se recree, las instantáneas permanecerán disponibles siempre que no se elimine el volumen persistente. En conjunto, este apartado documenta un mecanismo nativo de copia de seguridad en HBase y evidencia cómo su enfoque se deriva directamente de su modelo column-family y de su organización interna de almacenamiento, diferenciándose del paradigma relacional aplicado en PostgreSQL e IBM Db2.

6.5. Generación de datos y pruebas

El modelado en Apache HBase requiere un enfoque radicalmente distinto. Al ser una base de datos NoSQL orientada a columnas, no existen esquemas estrictos, tipos de datos complejos ni relaciones (JOIN). Todo el diseño debe girar en torno a la **RowKey** (clave de fila), ya que es el único elemento indexado por el que se puede buscar eficientemente.

Análisis de estrategias de generación de datos: HBase no posee un lenguaje SQL interactivo avanzado, sino una consola interactiva basada en Ruby. Por ello, las estrategias difieren:

1. **Importación mediante MapReduce (ImportTsv):** Es la herramienta nativa del ecosistema Hadoop para cargar ficheros CSV masivos, pero requiere levantar una infraestructura de clúster compleja.
2. **Generación mediante scripts en Python / Shell (estrategia seleccionada):** La forma más directa para pruebas de desarrollo es utilizar un script externo (en Python con la librería HappyBase, o bash) que genere secuencialmente los comandos put de HBase Shell.

Código de pruebas:

Para el caso de la biblioteca, se desnormalizan los datos en una única tabla. La **RowKey** será una concatenación del DNI del lector y la fecha, dando lugar a DNI-FECHA, para poder buscar fácilmente el historial de un usuario específico.

```
# Los siguientes comandos se ejecutan en la terminal HBase Shell

# 1. Definición del esquema (Solo se define el nombre de la tabla y su Familia de Columnas)
# 'info' será la familia de columnas que agrupará todos los datos del préstamo.
create 'historial_prestamos', 'info'

# 2. Estrategia de Inserción: Ejecución de comandos PUT generados externamente
# Sintaxis: put 'tabla', 'RowKey', 'familia:columna', 'valor'
put 'historial_prestamos', '12345678A-20260220', 'info:nombre_lector', 'Laura Gómez'
put 'historial_prestamos', '12345678A-20260220', 'info:titulo_libro', 'Cien años de soledad'

put 'historial_prestamos', '12345678A-20260225', 'info:nombre_lector', 'Laura Gómez'
put 'historial_prestamos', '12345678A-20260225', 'info:titulo_libro', 'El Quijote'

# 3. Consulta de prueba
# Como no hay sentencias SELECT ni WHERE, utilizamos un SCAN con un filtro de prefijo.
# Esto recupera todos los préstamos que empiecen por el DNI de Laura Gómez.
scan 'historial_prestamos', {ROWPREFIXFILTER => '12345678A'}
```

7. IBM Db2

7.1. Definición del servicio y control de versiones

Antes de proceder al despliegue de IBM Db2, se verificó el estado de los requisitos previos del entorno, comprobando tanto la versión de Docker Engine como la del plugin Docker Compose. Esta validación permite asegurar la compatibilidad del entorno con la imagen oficial del motor y garantizar la reproducibilidad del despliegue.

```
root@core ~/bd2/db2# docker --version
Docker version 29.2.1, build a5c7197
root@core ~/bd2/db2# docker compose version
Docker Compose version v5.0.2
```

La salida confirma que el sistema dispone de versiones recientes y operativas de Docker y Docker Compose, suficientes para gestionar contenedores complejos como el de IBM Db2.

Con el objetivo de mantener el entorno estructurado y fácilmente replicable, se creó un directorio de trabajo específico (~/.bd2/db2), destinado a contener tanto la definición declarativa del servicio como el volumen persistente asociado al motor.

```
root@core ~# mkdir -p ~/bd2/db2
root@core ~# cd ~/bd2/db2
```

El servicio de IBM Db2 se definió mediante un fichero docker-compose.yml, adoptando un enfoque declarativo que facilita la repetición del despliegue y la trazabilidad de la configuración frente a alternativas basadas en comandos extensos de tipo docker run. En la definición se empleó la imagen ibmcom/db2, se asignó un nombre explícito al contenedor (contenedor_db2) y se estableció una política de reinicio automático (restart: always) para asegurar su disponibilidad durante las pruebas.

Asimismo, se habilitó el modo privilegiado (privileged: true), requisito técnico de esta imagen para completar correctamente la inicialización del motor dentro del contenedor. En cuanto a la conectividad, se publicó el puerto estándar de IBM Db2 (50000), permitiendo conexiones externas hacia el gestor. Para garantizar la persistencia de los datos, se montó el volumen local ./data sobre el directorio interno /database, separando así el almacenamiento del ciclo de vida del contenedor.

```
services:
  db2:
    image: ibmcom/db2
    container_name: contenedor_db2
    restart: always
    privileged: true
    ports:
      - "50000:50000"
    environment:
      LICENSE: accept
      DB2INST1_PASSWORD: db2inst1
      DBNAME: db2inst1
      PERSISTENT_HOME: "false"
      ARCHIVE_LOGS: "false"
      TEXT_SEARCH: "false"
    volumes:
      - ./data:/database
```

Antes de iniciar el servicio, se creó el directorio local destinado a actuar como volumen persistente, asegurando que el almacenamiento estuviera preparado previamente al arranque del contenedor.

```
root@core ~/bd2/db2# mkdir -p data
```

Una vez definida la configuración, el servicio se desplegó en segundo plano mediante Docker Compose, lo que permitió aprovisionar automáticamente la red asociada, descargar la imagen y crear el contenedor conforme a la definición declarativa.

```
root@core ~/bd2/db2# docker compose up -d
[+] up 16/16
✓ Image ibmcom/db2          Pulled          114.5s
```

✓ Network db2_default	Created	0.2s
✓ Container contenedor_db2	Created	5.8s

Tras el despliegue, se comprobó que el contenedor se encontraba en ejecución y que el puerto 50000 estaba correctamente expuesto, lo que constituye evidencia de que el servicio quedó accesible desde el exterior del contenedor.

```
root@core ~/bd2/db2# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
0dd9ee0354bf   ibmcom/db2     "/var/db2_setup/lib/..." 39 seconds ago Up 33
seconds       22/tcp, 55000/tcp, 60006-60007/tcp, 0.0.0.0:50000->50000/tcp,
[::]:50000->50000/tcp   contenedor_db2
```

Finalmente, se revisaron los logs del contenedor con el fin de confirmar que la imagen había completado correctamente el proceso interno de inicialización del motor. Esta verificación es especialmente relevante en IBM Db2, ya que durante el primer arranque se realizan tareas automáticas como la creación de usuarios, la inicialización de la instancia, la aplicación de la licencia y la creación opcional de la base de datos inicial.

```
root@core ~/bd2/db2# docker logs -f contenedor_db2
(*) Previous setup has not been detected. Creating the users...
(*) Creating users ...
(*) Creating instance ...
DB2 installation is being initialized.
...
LIC1402I License added successfully.
LIC1426I This product is now licensed for use as outlined in your License
Agreement.
...
DB2 State : Operable
DB2 has not been started
Starting DB2...
02/18/2026 17:30:05 0 0 SQL1063N DB2START processing was successful.
SQL1063N DB2START processing was successful.
(*) User chose to create db2inst1 database
(*) Creating database db2inst1 ...
```

En la salida se observan mensajes que evidencian la creación de la instancia, la aceptación y aplicación de la licencia, el estado “Operable” del gestor, la ejecución satisfactoria de DB2START y la creación de la base de datos db2inst1. Estos registros constituyen prueba directa de que el motor ha sido correctamente inicializado y se encuentra en funcionamiento.

Con estas comprobaciones se valida que el servicio de IBM Db2 quedó desplegado de forma reproducible mediante Docker y Docker Compose, con persistencia configurada, puerto de servicio expuesto y control explícito del proceso de instalación y arranque a través de evidencias obtenidas desde la consola del sistema.

7.2. Creación y configuración inicial de la base de datos

Una vez verificado el despliegue del servicio de IBM Db2, se accedió al contenedor para completar la configuración inicial del entorno de trabajo desde el contexto adecuado. En Db2, las operaciones de administración y creación de bases de datos deben ejecutarse bajo el usuario de instancia, que actúa como identificador de autorización principal y es creado durante la inicialización automática del motor. Por ello, se abrió una sesión interactiva en el contenedor y se cambió al usuario db2inst1.

```
root@core ~/bd2/db2# docker exec -it contenedor_db2 bash
[root@0dd9ee0354bf /]# su - db2inst1
Last login: Wed Feb 18 17:30:05 UTC 2026
```

En caso de que el gestor no estuviese activo, podría iniciarse manualmente mediante db2start. En este despliegue, el motor ya se encontraba operativo, lo que se comprobó ejecutando el comando y observando el mensaje devuelto por el propio gestor.

```
[db2inst1@0dd9ee0354bf ~]$ db2start
02/18/2026 17:33:35 0 0 SQL1026N The database manager is already active.
SQL1026N The database manager is already active.
```

La respuesta SQL1026N The database manager is already active confirma que el database manager estaba iniciado, por lo que el entorno se encontraba listo para crear y gestionar bases de datos.

Aunque la imagen de Docker puede crear automáticamente una base de datos inicial a partir de la variable DBNAME, se optó por crear una base de datos específica para la práctica con el fin de trabajar sobre un entorno limpio y claramente identificado. Durante este proceso se observó una diferencia relevante respecto a otros SGBD. IBM Db2 impone restricciones estrictas sobre el nombre de la base de datos, que debe cumplir reglas formales (por ejemplo, longitud máxima y caracteres permitidos). Un primer intento con un nombre que no se ajustaba a dichas condiciones provocó un error, lo que permitió evidenciar esta restricción de forma explícita.

```
[db2inst1@0dd9ee0354bf ~]$ db2 create database practica_db2
SQL1001N "practica_db2" is not a valid database name.  SQLSTATE=2E000
```

El mensaje SQL1001N "<nombre>" is not a valid database name confirma que el identificador propuesto no era válido según las reglas de nombrado de Db2. Tras ajustar el nombre a un identificador conforme, se creó correctamente la base de datos PRACTDB2.

```
[db2inst1@0dd9ee0354bf ~]$ db2 create database PRACTDB2
DB20000I The CREATE DATABASE command completed successfully.
```

Para verificar su disponibilidad y confirmar la versión efectiva del motor, se estableció conexión con la nueva base de datos mediante db2 connect.

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2
Database Connection Information
Database server      = DB2/LINUX8664 11.5.8.0
SQL authorization ID = DB2INST1
Local database alias = PRACTDB2
```

La información mostrada en la conexión confirma, por un lado, la versión del servidor DB2/LINUX8664 11.5.8.0 y, por otro, que la sesión se establece con el identificador de autorización DB2INST1, que será el propietario por defecto de los objetos creados si no se especifica otro esquema o autorización. Con ello queda configurada una base de datos específica para la práctica, independiente de cualquier base creada automáticamente durante el arranque del contenedor, permitiendo aplicar de forma controlada la configuración de seguridad, la creación de roles/usuarios y las pruebas funcionales desarrolladas en los apartados siguientes.

7.3. Configuración de seguridad y gestión de roles

Una vez creada la base de datos PRACTDB2, se configuró el modelo de seguridad con el objetivo de definir perfiles diferenciados de acceso, en línea con los requisitos de la práctica relativos a la creación de usuarios con distintos niveles de privilegio. En IBM Db2, la autorización se articula en torno a authorization IDs, que en entornos Linux suelen corresponderse con usuarios del sistema operativo. Sobre estos identificadores pueden definirse roles dentro del motor, conceder privilegios a dichos roles (a nivel de base de datos y de objetos) y, finalmente, asignar los roles a usuarios concretos, facilitando una administración más mantenible que la asignación directa de privilegios usuario a usuario.

Con el fin de separar un perfil orientado a consulta de otro con capacidad de modificación, se definieron dos roles, ROL_LECTURA y ROL_ESCRITURA. La creación se realizó desde el usuario de instancia db2inst1, conectado a la base de datos de trabajo.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "CREATE ROLE ROL_LECTURA"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "CREATE ROLE ROL_ESCRITURA"
DB20000I The SQL command completed successfully.
```

A continuación, se aplicó una configuración inicial de privilegios a nivel de base de datos. En ambos roles se concedió CONNECT, necesario para permitir la conexión. Además, se concedió DATAACCESS como permiso general de acceso a datos. Para el rol de escritura se añadió también DBADM, con el objetivo de disponer de capacidades administrativas sobre la base de datos durante la práctica.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT CONNECT ON DATABASE TO ROLE ROL_LECTURA"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT DATAACCESS ON DATABASE TO ROLE
ROL_LECTURA"
DB20000I The SQL command completed successfully.
```

```
[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT CONNECT ON DATABASE TO ROLE ROL_ESCRITURA"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT DATAACCESS ON DATABASE TO ROLE
ROL_ESCRITURA"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT DBADM ON DATABASE TO ROLE ROL_ESCRITURA"
DB20000I The SQL command completed successfully.
```

En este punto se observó un matiz importante que condiciona la separación efectiva entre perfiles. En IBM Db2 el privilegio DATAACCESS no se limita a operaciones de lectura, sino que habilita también operaciones de manipulación de datos (INSERT, UPDATE y DELETE) sobre los objetos accesibles. Por tanto, aunque su denominación pueda sugerir un uso “de consulta”, en la práctica puede resultar demasiado amplio si el objetivo es construir un rol estrictamente de solo lectura. Este aspecto se tuvo en cuenta para las comprobaciones posteriores y para el posible ajuste fino de privilegios a nivel de objeto.

En el despliegue utilizado, la autenticación se apoya en usuarios del sistema operativo dentro del propio contenedor Linux. Por ello, para simular perfiles diferenciados se crearon dos usuarios del sistema, consultor y editor, asociados respectivamente a los roles de lectura y escritura. La creación se realizó desde el usuario root en el contenedor, asignando además credenciales para permitir el inicio de sesión.

```
[root@0dd9ee0354bf /]# useradd -m -s /bin/bash consultor
[root@0dd9ee0354bf /]# echo "consultor:consultor123" | chpasswd

[root@0dd9ee0354bf /]# useradd -m -s /bin/bash editor
[root@0dd9ee0354bf /]# echo "editor:editor123" | chpasswd
```

Posteriormente, desde la sesión del usuario de instancia db2inst1 y con conexión activa a PRACTDB2, se asignaron los roles definidos a cada uno de los usuarios creados.

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2
[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT ROLE ROL_LECTURA TO USER CONSULTOR"
DB20000I The SQL command completed successfully.
[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT ROLE ROL_ESCRITURA TO USER EDITOR"
DB20000I The SQL command completed successfully.
```

De este modo, el modelo de seguridad quedó estructurado en tres niveles. En primer lugar, los usuarios del sistema operativo (consultor y editor) actúan como identidades autenticables en el entorno contenedor. En segundo lugar, los roles del motor (ROL_LECTURA y ROL_ESCRITURA) encapsulan la política de permisos. En tercer lugar, los privilegios se conceden a los roles, evitando duplicación y permitiendo ajustar la política de acceso de forma centralizada. En el subapartado siguiente se validará experimentalmente el comportamiento resultante y, si es necesario, se ajustará la asignación de privilegios para garantizar una separación real entre operaciones de consulta y de modificación.

7.4. Estrategia de copia de seguridad

Una vez configurada la base de datos y verificado su acceso local y remoto, se definió y ejecutó una estrategia de copia de seguridad utilizando mecanismos nativos de IBM Db2. Aunque el entorno de virtualización permite crear snapshots de la máquina virtual completa, este enfoque no se considera una copia de seguridad válida desde el punto de vista del SGBD. Un snapshot captura el estado global del sistema (SO, configuración y datos), pero no genera un respaldo gestionado por el motor, no facilita restauraciones selectivas a nivel de base de datos y tampoco constituye evidencia del uso de herramientas propias del gestor. Por ello, desde una perspectiva académica y profesional, se empleó el mecanismo nativo de Db2.

IBM Db2 proporciona el comando `BACKUP DATABASE`, que genera una copia física consistente de la base de datos. Dado que el contenedor se configuró con persistencia mediante volumen (./data montado sobre /database), se decidió almacenar el backup en ese directorio para garantizar que el artefacto de respaldo quedase fuera del ciclo de vida del contenedor. En primer lugar, se estableció conexión con la base de datos `PRACTDB2` desde el usuario de instancia.

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2
Database Connection Information
Database server      = DB2/LINUX8664 11.5.8.0
SQL authorization ID = DB2INST1
Local database alias = PRACTDB2
```

A continuación, se intentó ejecutar la copia de seguridad hacia el directorio persistente.

```
[db2inst1@0dd9ee0354bf ~]$ db2 backup database PRACTDB2 to /database
SQL2061N  An attempt to access media "/database" is denied.
```

El gestor devolvió el error `SQL2061N`, indicando que el acceso al medio de destino estaba denegado. Este resultado apunta a un problema de permisos a nivel de sistema de archivos. En este despliegue, `/database` corresponde a un volumen montado desde el host, creado inicialmente con propietario `root`. Al ejecutar el backup como `db2inst1`, dicho usuario no disponía de permisos de escritura sobre el directorio, por lo que Db2 no pudo crear la imagen de backup.

Para verificar esta hipótesis y corregirla, se revisaron los permisos del directorio y se ajustó el propietario para que el usuario de instancia pudiera escribir en el volumen persistente.

```
[root@0dd9ee0354bf /]# ls -ld /database
drwxr-xr-x 4 root root 4096 Feb 18 17:29 /database
[root@0dd9ee0354bf /]# chown -R db2inst1:db2iadm1 /database
[root@0dd9ee0354bf /]# chmod -R u+rwX /database
```

Una vez corregidos los permisos, se repitió el proceso desde el usuario `db2inst1` y el backup se completó correctamente.

```
[root@0dd9ee0354bf /]# su - db2inst1
```

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2
[db2inst1@0dd9ee0354bf ~]$ db2 backup database PRACTDB2 to /database
Backup successful. The timestamp for this backup image is : 20260218182327
```

El mensaje final de éxito confirma que se generó una imagen física de la base de datos, asociada a una marca temporal. Al haberse almacenado en /database (volumen persistente), el backup permanece disponible en el host incluso si el contenedor se detiene o se elimina. Con esta operación se valida el uso de la herramienta nativa de IBM Db2 para copias de seguridad, cumpliendo los requisitos de administración del SGBD y evidenciando además la interacción entre el motor Db2, los permisos del sistema operativo del contenedor y la persistencia gestionada mediante Docker.

7.5. Generación de datos y pruebas

7.5.1. Pruebas funcionales y validación de roles

Una vez configurada la base de datos PRACTDB2 y aplicado el esquema de roles definido en el apartado anterior, se realizaron pruebas funcionales con un doble objetivo. En primer lugar, validar el funcionamiento básico mediante creación de tablas e inserción/consulta de datos. En segundo lugar, comprobar experimentalmente que la separación de privilegios entre usuarios era efectiva y, en caso contrario, ajustar la política de permisos.

Para disponer de un objeto de pruebas sencillo y controlado, se creó una tabla mínima con una clave primaria autogenerada y un campo de texto. Esta operación se realizó como usuario de instancia DB2INST1, que actúa como propietario por defecto de los objetos creados en el esquema.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "CREATE TABLE PRUEBAS_FINAL (
    ID INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    DATO VARCHAR(100)
)"
DB20000I  The SQL command completed successfully.
```

A continuación, se insertaron dos registros y se verificó su persistencia mediante una consulta de selección, confirmando que la tabla era operativa y que el gestor almacenaba correctamente los cambios.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "INSERT INTO PRUEBAS_FINAL (DATO) VALUES ('Dato
visible 1')"
```

```
DB20000I  The SQL command completed successfully.
```

```
[db2inst1@0dd9ee0354bf ~]$ db2 "INSERT INTO PRUEBAS_FINAL (DATO) VALUES ('Dato
visible 2')"
```

```
DB20000I  The SQL command completed successfully.
```

```
[db2inst1@0dd9ee0354bf ~]$ db2 "SELECT * FROM PRUEBAS_FINAL"
```

ID	DATO
1	Dato visible 1
2	Dato visible 2

```
1          Dato visible 1
2          Dato visible 2
2 record(s) selected.
```

Dado que la tabla fue creada por DB2INST1, su nombre completo queda registrado como DB2INST1.PRUEBAS_FINAL. Este detalle resultó relevante durante las pruebas posteriores, ya que en IBM Db2 la resolución de objetos depende del esquema asociado al usuario que ejecuta la consulta.

Seguidamente, se probó el acceso con el usuario consultor, previsto como perfil de solo lectura. La conexión se estableció correctamente, confirmando que el usuario podía autenticarse contra la base de datos.

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2 user consultor using
consultor123
Database Connection Information
Database server      = DB2/LINUX8664 11.5.8.0
SQL authorization ID = CONSULTO...
Local database alias = PRACTDB2
```

Sin embargo, al ejecutar una consulta sin especificar el esquema, Db2 devolvió un error indicando que el objeto no existía bajo el esquema del usuario actual.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "SELECT * FROM PRUEBAS_FINAL"
SQL0204N  "CONSULTOR.PRUEBAS_FINAL" is an undefined name.  SQLSTATE=42704
```

El error SQL0204N "<esquema>.PRUEBAS_FINAL is an undefined name" no estaba relacionado con privilegios, sino con la gestión de esquemas. Por defecto, cada usuario dispone de un esquema implícito con su mismo nombre, por lo que una referencia sin calificar se interpreta internamente como CONSULTOR.PRUEBAS_FINAL. Al haberse creado la tabla en el esquema DB2INST1, la consulta correcta debe referenciar explícitamente el esquema propietario.

En este contexto, se detectó además un problema de seguridad. Aunque consultor debía ser únicamente lector, se comprobó que podía insertar datos cuando se indicaba el nombre completo de la tabla.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "INSERT INTO DB2INST1.PRUEBAS_FINAL (DATO) VALUES
('Intento hacker')"
DB20000I  The SQL command completed successfully.
```

La causa fue la concesión inicial del privilegio DATAACCESS al rol ROL_LECTURA. En IBM Db2, DATAACCESS no se limita a lectura, sino que habilita también operaciones DML (INSERT, UPDATE y DELETE) sobre los objetos accesibles. Esta observación evidenció que, si se busca una separación estricta entre lectura y escritura, resulta preferible definir permisos de forma más granular a nivel de objeto (por ejemplo, SELECT sobre tablas concretas) en lugar de utilizar privilegios amplios a nivel de base de datos.

Para corregirlo, se revocó DATAACCESS del rol de lectura y se concedió únicamente SELECT sobre la tabla de pruebas.

```
[db2inst1@0dd9ee0354bf ~]$ db2 "REVOKE DATAACCESS ON DATABASE FROM ROLE
ROL_LECTURA"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "GRANT SELECT ON TABLE DB2INST1.PRUEBAS_FINAL TO
ROLE ROL_LECTURA"
DB20000I The SQL command completed successfully.
```

Tras el ajuste, se repitió la prueba de inserción con el usuario consultor y Db2 rechazó la operación con el error esperado, confirmando que el acceso quedaba efectivamente restringido a lectura.

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2 user consultor using
consultor123

[db2inst1@0dd9ee0354bf ~]$ db2 "INSERT INTO DB2INST1.PRUEBAS_FINAL (DATO) VALUES
('Intento hacker')"
SQL0551N The statement failed because the authorization ID does not have the
required authorization or privilege to perform the operation.
SQLSTATE=42501
```

El error SQL0551N indica que el identificador de autorización no dispone del privilegio requerido para ejecutar la operación, lo que valida la corrección aplicada al modelo de permisos.

Finalmente, se verificó el perfil con permisos de escritura utilizando el usuario editor, asociado al rol ROL_ESCRITURA. La conexión se realizó correctamente y se comprobaron operaciones de inserción, actualización y borrado sobre la tabla, evidenciando que el perfil dispone de permisos de modificación coherentes con su propósito.

```
[db2inst1@0dd9ee0354bf ~]$ db2 connect to PRACTDB2 user editor using editor123

[db2inst1@0dd9ee0354bf ~]$ db2 "INSERT INTO DB2INST1.PRUEBAS_FINAL (DATO) VALUES
('Inserción permitida')"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "UPDATE DB2INST1.PRUEBAS_FINAL SET DATO =
'Modificado por editor' WHERE ID = 1"
DB20000I The SQL command completed successfully.

[db2inst1@0dd9ee0354bf ~]$ db2 "DELETE FROM DB2INST1.PRUEBAS_FINAL WHERE DATO =
'Inserción permitida'"
DB20000I The SQL command completed successfully.
```

En conjunto, estas pruebas validan el funcionamiento básico del gestor y confirman que el modelo de seguridad finalmente aplicado garantiza una separación efectiva entre usuarios con permisos estrictamente de solo lectura y usuarios con capacidad de modificación de

datos. Además, el proceso seguido documenta un ciclo completo de verificación y ajuste, identificando un comportamiento no deseado derivado de un privilegio excesivamente amplio y corrigiéndolo mediante revocación y concesión selectiva a nivel de objeto.

7.5.2. Implementación de la generación de datos

Al tratarse de un SGBD relacional de nivel empresarial, IBM Db2 permite implementar el mismo esquema normalizado de la biblioteca utilizado en PostgreSQL y Oracle, respetando la integridad referencial mediante claves primarias y foráneas.

Análisis de estrategias de generación de datos: Para la ingesta masiva de datos en entornos corporativos con Db2, se han evaluado dos enfoques principales:

1. **Procedimientos almacenados (SQL PL):** Al igual que Oracle, Db2 soporta lenguajes procedimentales internos para generar datos en el servidor, aunque su sintaxis es más estricta.
2. **Utilidades nativas de carga masiva (IMPORT / LOAD) (estrategia seleccionada):** Db2 destaca por sus utilidades de ingesta de datos de alto rendimiento. La estrategia más realista en este entorno es utilizar una herramienta externa (como Mockaroo) para generar un fichero CSV con miles de registros ficticios, y utilizar la herramienta nativa IMPORT desde la línea de comandos para poblar las tablas.

Código de pruebas:

```
-- 1. Definición del esquema relacional (Sintaxis específica de Db2)
CREATE TABLE lectores (
    dni VARCHAR(15) NOT NULL PRIMARY KEY,
    nombre VARCHAR(50)
);

CREATE TABLE libros (
    isbn VARCHAR(20) NOT NULL PRIMARY KEY,
    titulo VARCHAR(100)
);

CREATE TABLE prestamos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dni_lector VARCHAR(15) REFERENCES lectores(dni),
    isbn_libro VARCHAR(20) REFERENCES libros(isbn),
    fecha_prestamo DATE
);

-- 2. Inserción de catálogo base
INSERT INTO lectores (dni, nombre) VALUES ('12345678A', 'Laura Gómez');
INSERT INTO libros (isbn, titulo) VALUES ('978-84', 'Cien años de soledad');

-- 3. Estrategia de ingesta: Carga masiva mediante utilidad nativa IMPORT
-- (Este comando se ejecuta desde la consola del cliente Db2, no como SQL
```

```

estándar)
-- IMPORT FROM prestamos_generados.csv OF DEL INSERT INTO prestamos;

-- 4. Inserción manual de prueba para validar la restricción
INSERT INTO prestamos (dni_lector, isbn_libro, fecha_prestamo)
VALUES ('12345678A', '978-84', CURRENT DATE);

-- 5. Consulta de prueba (Verificación de relaciones mediante JOIN)
SELECT l.nombre AS Lector, b.titulo AS Libro, p.fecha_prestamo
FROM prestamos p
JOIN lectores l ON p.dni_lector = l.dni
JOIN libros b ON p.isbn_libro = b.isbn;

```

8. Configuración de acceso remoto a los SGBD

Con el objetivo de cumplir el requisito establecido en el enunciado de la práctica, se diseñó una estrategia que permitiera el acceso remoto a los distintos SGBD desplegados en la máquina virtual del equipo, garantizando al mismo tiempo estabilidad y reproducibilidad del entorno. El entorno de trabajo presenta la siguiente arquitectura:

- Máquina física anfitriona (Windows).
- Máquina virtual Linux ejecutada en VirtualBox.
- Contenedores Docker que alojan los distintos SGBD.
- Configuración de red basada en NAT con reenvío de puertos.

Las pruebas de conectividad se realizaron desde un segundo equipo físico conectado a la misma red Wi-Fi. En nuestro caso, la red empleada correspondía a la red institucional de la Universidad de Zaragoza, lo cual se infirió a partir de los parámetros observados en el equipo anfitrión: (i) la dirección IPv4 asignada pertenecía al rango privado 10.0.0.0/8 (por ejemplo, 10.1.65.71), (ii) la puerta de enlace y el servidor DHCP se encontraban en el mismo rango (10.1.95.254) y (iii) los servidores DNS configurados (155.210.12.13 y 155.210.12.14) resolvían el dominio *unizar.es*, lo que confirma el uso de infraestructura DNS institucional.

1. Extracto de **ipconfig /all** (solo el bloque Wi-Fi)

Adaptador de LAN inalámbrica Wi-Fi:

```

Dirección IPv4. . . . . : 10.1.65.71(Preferido)
Puerta de enlace predeterminada . . . . . : 10.1.95.254
Servidor DHCP . . . . . : 10.1.95.254
Servidores DNS. . . . . : 155.210.12.13
                        155.210.12.14

```

2. Extracto de **nslookup unizar.es**

```

Servidor: UnKnown
Address: 155.210.12.13

```

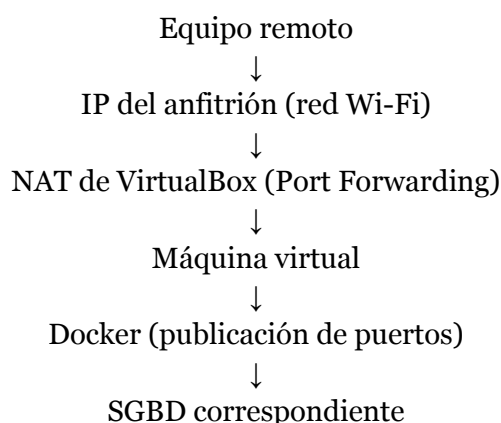
Respuesta no autoritativa:

Nombre: unizar.es Address: 155.210.11.203
--

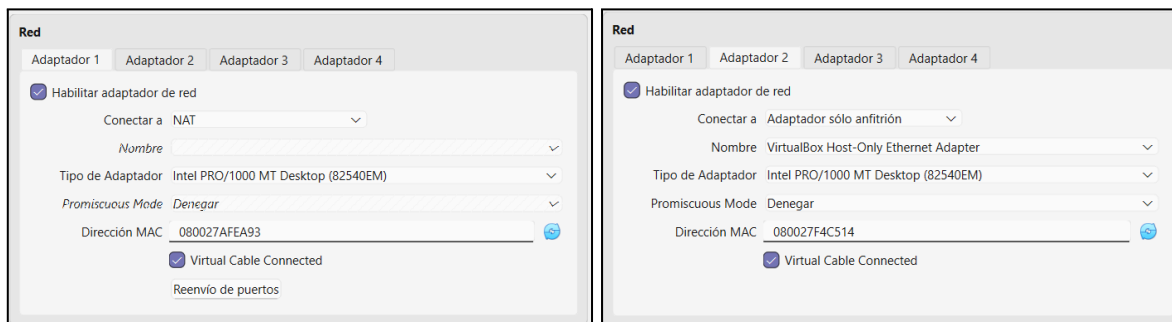
Dado que la dirección IP del equipo anfitrión se obtiene mediante DHCP, su valor puede variar entre sesiones (por ejemplo, tras reinicios, desconexiones o cambios de red). Por este motivo, en los ejemplos de los subapartados siguientes se utilizará la notación `<IP_DEL_ANFITRION>` para referirse a la dirección IPv4 activa del equipo anfitrión en la red local, por lo que antes de cada prueba se verificó con el comando *ipconfig*.

Desde el punto de vista de red, el acceso remoto implica que se cumplan simultáneamente varias condiciones: (1) el servicio del SGBD debe estar en ejecución y escuchar en una interfaz accesible; (2) el puerto correspondiente debe estar publicado correctamente desde el contenedor Docker hacia la máquina virtual; (3) debe existir una regla de reenvío de puertos en VirtualBox que vincule el puerto del anfitrión con el puerto de la máquina virtual; y (4) no deben existir restricciones de firewall que bloqueen la comunicación entrante.

La utilización combinada de VirtualBox y Docker introduce múltiples capas de virtualización, por lo que fue necesario verificar cuidadosamente el encadenamiento correcto de puertos, que en nuestro entorno se resume en:



Se configuraron reglas de reenvío para los siguientes servicios: PostgreSQL (5432), Oracle XE (1521), Apache Cassandra (9042), Apache HBase (interfaz web, 16010) e IBM Db2 (50000). Un aspecto crítico consistió en dejar vacío el campo “IP anfitrión” en las reglas de port forwarding, de forma que el sistema escuchase en todas las interfaces (0.0.0.0) y permitiera conexiones externas, en lugar de limitarse a 127.0.0.1. Para cada SGBD se siguió el mismo procedimiento de validación: primero se comprobó la accesibilidad del puerto desde el segundo equipo, y posteriormente se confirmó la conexión mediante el cliente correspondiente, tal como se detalla en los subapartados 8.1–8.5.



Figuras 9 y 10. Configuración de red de la máquina virtual en VirtualBox

8.1. PostgreSQL

Una vez configurado el reenvío de puertos en el adaptador NAT de VirtualBox y verificada la correcta publicación del puerto 5432 en Docker, se procedió a validar el acceso al servicio PostgreSQL mediante dos evidencias complementarias: (i) conexión mediante cliente gráfico (DBeaver) desde el equipo anfitrión y (ii) verificación de accesibilidad desde un segundo equipo físico en la red local.

En primer lugar, se verificó el acceso mediante DBeaver desde el equipo anfitrión (Windows), lo que permite comprobar de forma práctica la correcta exposición del servicio y la operativa de autenticación y consulta. Para ello, se creó una nueva conexión PostgreSQL en DBeaver (Figura 11) y se configuraron los parámetros generales manteniendo el resto de opciones por defecto:

- **Host:** localhost
- **Puerto:** 5432
- **Database:** practica1_db
- **Usuario:** alumno
- **Contraseña:** alumno123

Adicionalmente, en la pestaña **Advanced** se cambió el valor de **Local Client** a **PostgreSQL Binaries**. Tras finalizar el asistente, la conexión quedó registrada en el panel lateral de DBeaver, desde donde se pudo acceder a la base de datos y ejecutar consultas. Las Figuras 12–16 muestran la conexión establecida y la visualización de la tabla `pruebas_final`, confirmando que el servicio acepta conexiones TCP/IP y que las credenciales definidas funcionan correctamente.

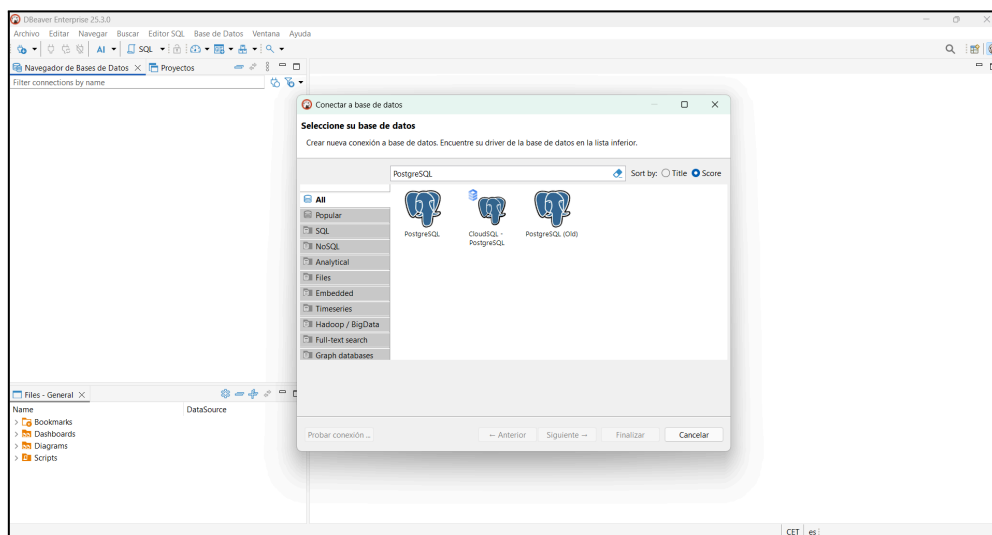
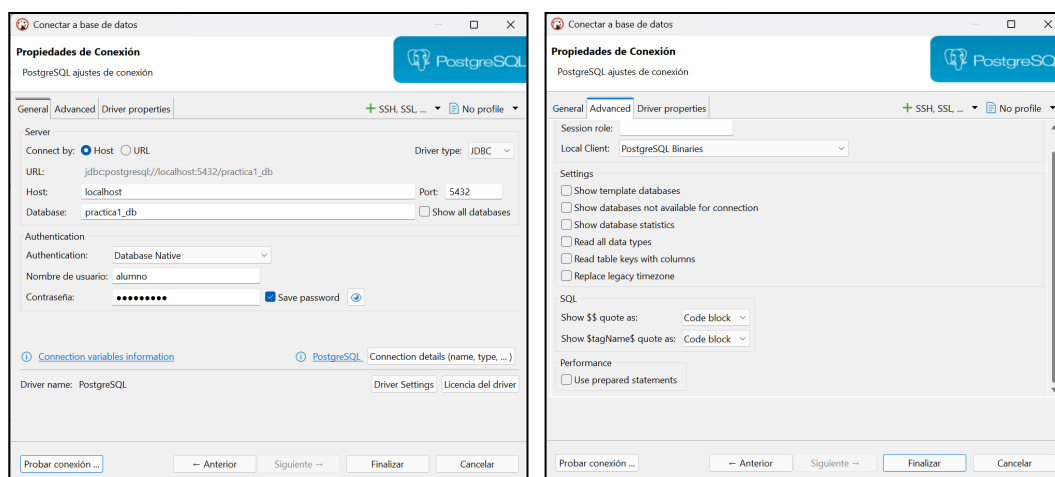


Figura 11. Nueva conexión PostgreSQL en DBeaver



Figuras 12 y 13. Configuración de la nueva base de datos PostgreSQL

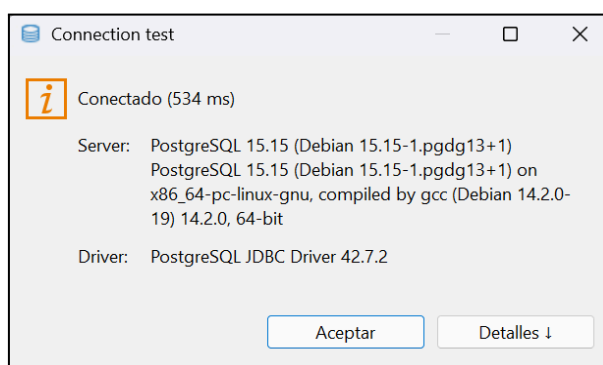


Figura 14. Conexión exitosa en DBeaver

Tras finalizar el asistente, la conexión quedó registrada en el panel lateral izquierdo de DBeaver, desde donde se pudo acceder a la base de datos y ejecutar consultas.

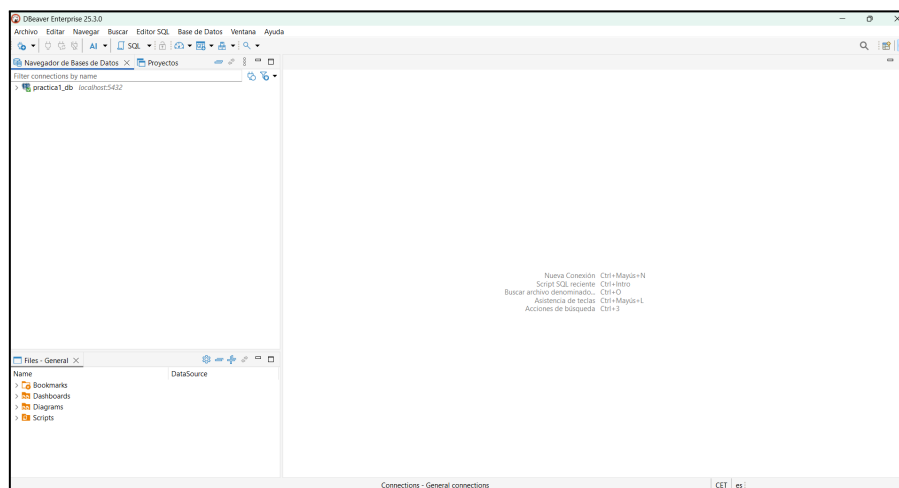


Figura 15. Conexión PostgreSQL en pantalla principal

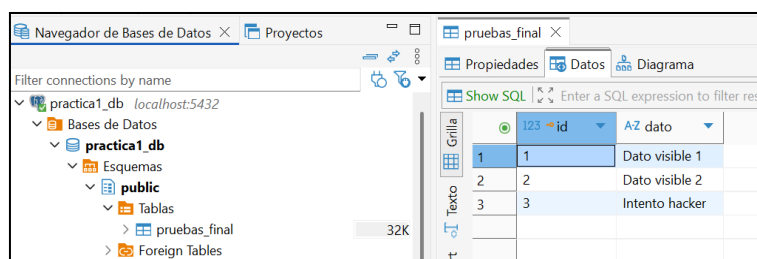


Figura 16. Visualización de la tabla pruebas_final en local (DBeaver)

A continuación, con objeto de cumplir el requisito de acceso remoto desde un equipo físico distinto, se comprobó la accesibilidad del puerto 5432 desde un segundo equipo conectado a la misma red Wi-Fi. Desde dicho equipo se ejecutó en PowerShell el comando:

```
Test-NetConnection <IP_DEL_ANFITRION> -Port 5432
```

donde <IP_DEL_ANFITRION> corresponde a la dirección IPv4 del equipo anfitrión en la red local. Obteniéndose como resultado:

```
TcpTestSucceeded : True
```

Este resultado confirma que:

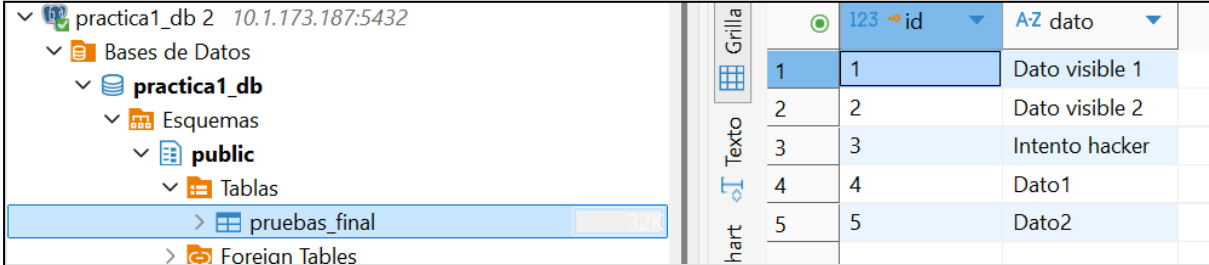
- El puerto 5432 está accesible desde el exterior.
- El reenvío de puertos configurado en VirtualBox funciona correctamente.
- No existen restricciones de firewall que bloqueen la conexión.
- La comunicación a nivel de red entre ambos equipos es viable.

Superada la validación a nivel de red, se estableció conexión mediante cliente gráfico (DBeaver), utilizando los siguientes parámetros:

- Host: <IP_DEL_ANFITRION>
- Port: 5432
- Base de datos: practica1_db
- Usuario: alumno

- Contraseña: alumno123

La Figura 17 muestra la conexión remota activa y la visualización de la tabla pruebas_final dentro de la base de datos practica1_db, donde se observan los registros insertados previamente.



	id	dato
1	1	Dato visible 1
2	2	Dato visible 2
3	3	Intento hacker
4	4	Dato1
5	5	Dato2

Figura 17. Visualización de la tabla pruebas_final en remoto (DBeaver)

La correcta visualización de los datos confirma que:

- El tráfico entrante es redirigido desde el equipo anfitrión hacia la máquina virtual mediante NAT.
- Docker publica correctamente el puerto del contenedor.
- El servicio PostgreSQL acepta conexiones remotas.
- Las credenciales y permisos definidos funcionan adecuadamente en un escenario distribuido.

Esta prueba valida de forma completa el cumplimiento del requisito de acceso remoto desde un equipo físico distinto al que aloja el servidor.

8.2. Oracle XE

Tras la configuración del reenvío de puertos correspondiente al servicio de Oracle Database Express Edition (puerto 1521) en el adaptador NAT de VirtualBox, se procedió a validar el acceso al listener de Oracle mediante dos evidencias complementarias: (i) conexión mediante cliente gráfico (DBeaver) desde el equipo anfitrión y (ii) verificación de accesibilidad desde un segundo equipo físico conectado a la misma red local.

En primer lugar, se registró la conexión en DBeaver desde el equipo anfitrión (Windows), lo que permite comprobar de forma práctica que el listener está operativo y que la autenticación funciona correctamente a través del puerto expuesto. Para ello, se configuró una nueva conexión Oracle en DBeaver con los parámetros mostrados en la Figura 18, utilizando:

- **Host:** localhost
- **Puerto:** 1521
- **Database:** XEPDB1
- **Usuario:** alumno
- **Contraseña:** alumno123

Las Figuras 19–20 muestran la conexión establecida y el acceso al esquema correspondiente, confirmando que el listener está activo, la base de datos se encuentra operativa y las

credenciales configuradas permiten el acceso desde el cliente JDBC una vez publicado el puerto.

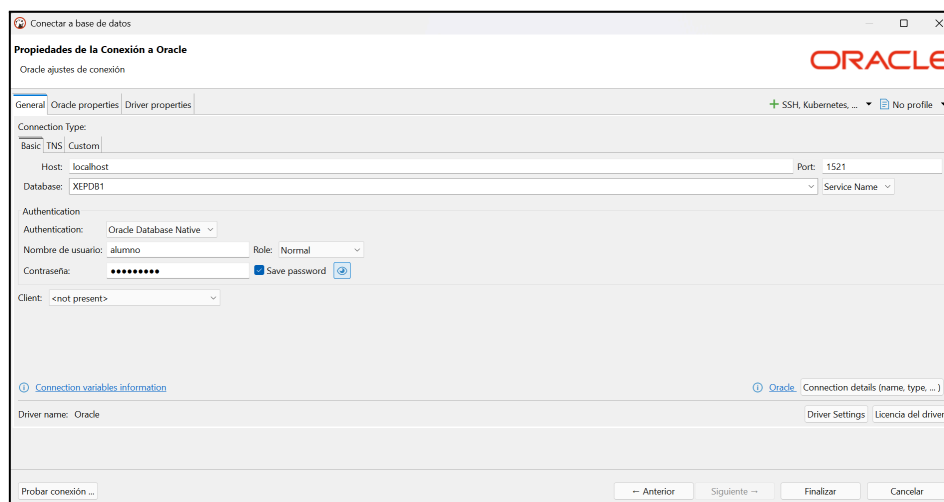


Figura 18. Configuración de la nueva base de datos Oracle

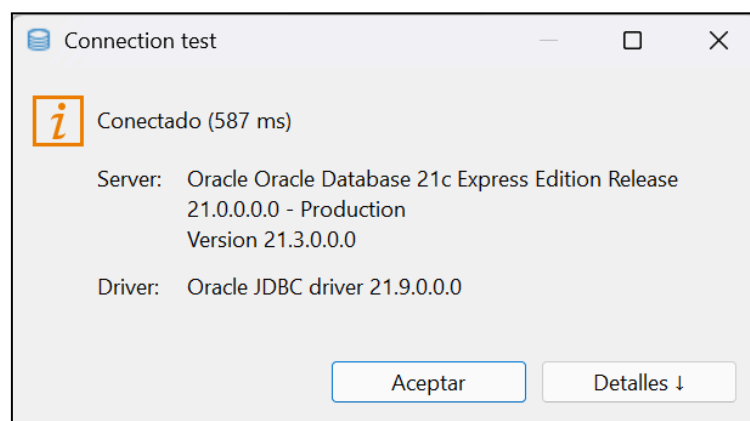


Figura 19. Conexión exitosa en DBeaver

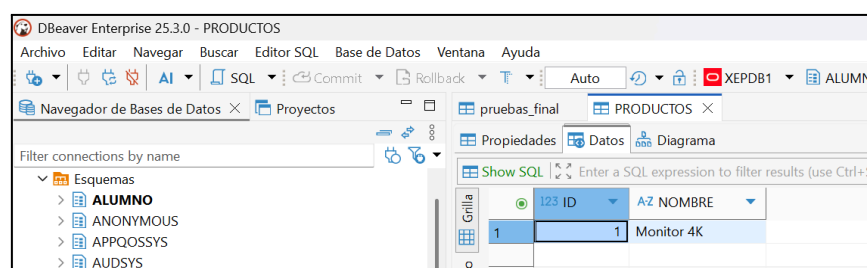


Figura 20. Visualización de la tabla PRODUCTOS en local (DBeaver)

A continuación, con objeto de verificar la accesibilidad remota desde un equipo físico distinto, se comprobó el acceso al puerto 1521 desde un segundo equipo conectado a la misma red Wi-Fi. Desde dicho equipo se ejecutó en PowerShell el siguiente comando:

```
Test-NetConnection <IP_DEL_ANFITRION> -Port 1521
```

donde <IP_DEL_ANFITRION> corresponde a la dirección IPv4 del equipo anfitrión en la red Wi-Fi. Obteniéndose como resultado:

TcpTestSucceeded : True

lo que confirmaría que:

- El puerto 1521 está accesible desde el exterior.
- El reenvío de puertos en VirtualBox está correctamente configurado.
- No existen bloqueos a nivel de firewall.
- La red permite comunicación directa entre dispositivos.

Una vez validada la conectividad a nivel de red, se procede a establecer conexión mediante cliente gráfico (DBeaver), utilizando los siguientes parámetros:

- Host: <IP_DEL_ANFITRION>
- Port: 1521
- Service Name: XEPDB1
- Usuario: alumno
- Contraseña: alumno123

La correcta conexión y visualización de tablas dentro del esquema correspondiente confirmará que:

- El listener de Oracle está activo.
- La base de datos está operativa.
- Las credenciales configuradas permiten acceso remoto.
- La arquitectura NAT + Port Forwarding funciona de forma consistente también para Oracle XE.

Se incluirá una captura de pantalla (Figura 21) mostrando la conexión remota establecida y la consulta de datos desde el segundo equipo físico, como evidencia del correcto funcionamiento.

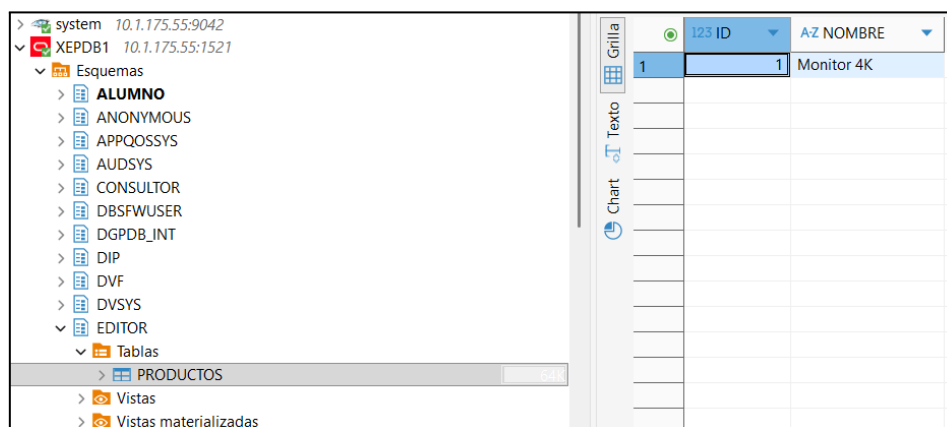


Figura 21. Visualización de la tabla *PRODUCTOS* en remoto (DBeaver)

8.3. Apache Cassandra

Una vez configurado el reenvío de puertos correspondiente al servicio de Apache Cassandra (puerto 9042) en el adaptador NAT de VirtualBox, se procedió a validar el acceso al servicio CQL (Cassandra Query Language) mediante dos evidencias complementarias: (i) conexión

mediante cliente gráfico (DBeaver) desde el equipo anfitrión y (ii) verificación de accesibilidad desde un segundo equipo físico conectado a la misma red local.

A diferencia de los SGBD relacionales, Cassandra expone un servicio CQL accesible a través del puerto 9042, por lo que la validación consiste en comprobar la conectividad hacia dicho puerto y verificar posteriormente el acceso a los datos mediante un cliente compatible.

En primer lugar, se configuró la conexión en DBeaver desde el equipo anfitrión, utilizando los parámetros mostrados en la Figura 22 (manteniendo el resto de opciones por defecto). En particular, se estableció la conexión hacia el puerto CQL (9042) y se accedió al keyspace de trabajo `practica1_keyspace`, pudiéndose explorar la tabla `productos`. Las Figuras 23–24 muestran la conexión establecida y la visualización de los registros insertados previamente, lo que confirma que el servicio Cassandra está operativo, que el puerto CQL se encuentra correctamente publicado y que la estructura de datos definida en el keyspace es accesible desde el cliente.

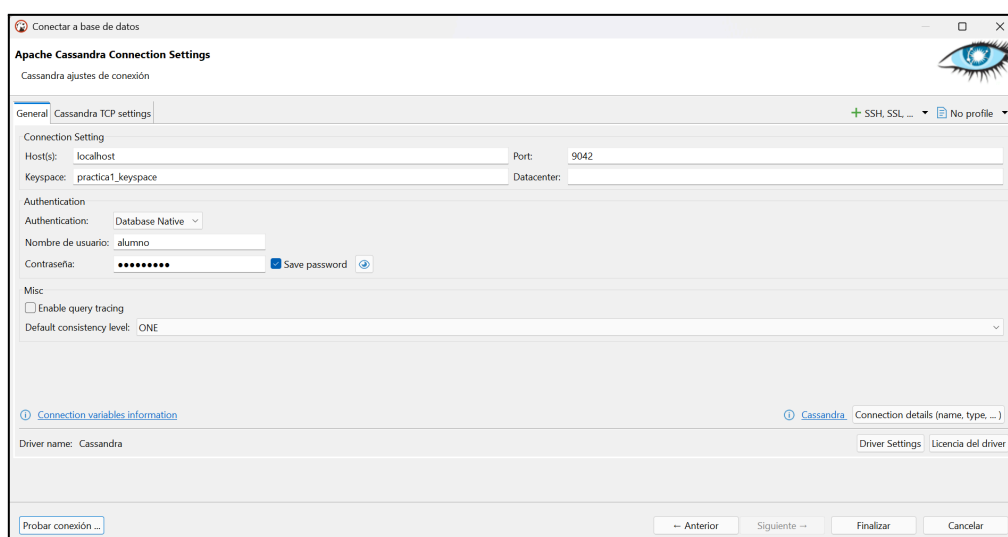


Figura 22. Configuración de la nueva base de datos Apache Cassandra

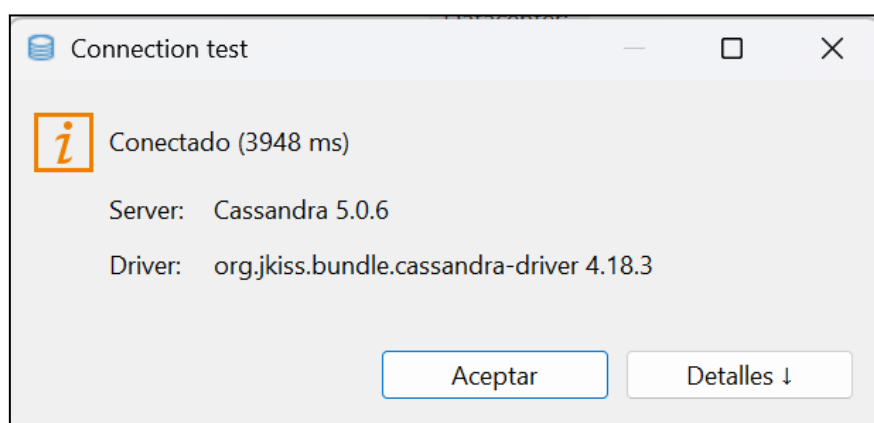


Figura 23. Conexión exitosa en DBeaver

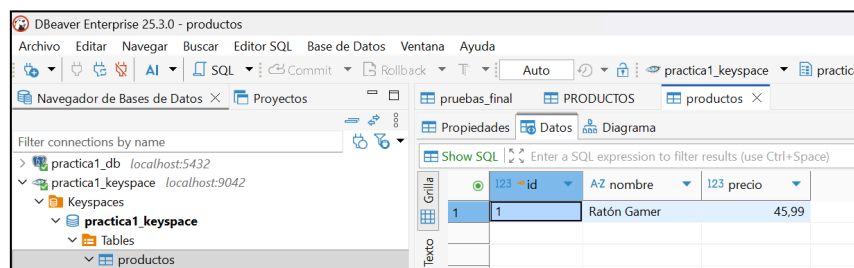


Figura 24. Visualización de la tabla productos en local (DBeaver)

A continuación, con objeto de verificar la accesibilidad remota desde un equipo físico distinto, se comprobó el acceso al puerto 9042 desde un segundo equipo conectado a la misma red Wi-Fi. Desde dicho equipo se ejecutó en PowerShell el comando:

```
Test-NetConnection <IP_DEL_ANFITRION> -Port 9042
```

donde <IP_DEL_ANFITRION> corresponde a la dirección IPv4 del equipo anfitrión en la red Wi-Fi. Obteniéndose como resultado:

```
TcpTestSucceeded : True
```

lo que confirma que:

- El puerto 9042 está accesible desde el exterior.
- El reenvío de puertos en VirtualBox funciona correctamente.
- No existen bloqueos a nivel de firewall.
- La comunicación entre ambos equipos es viable.

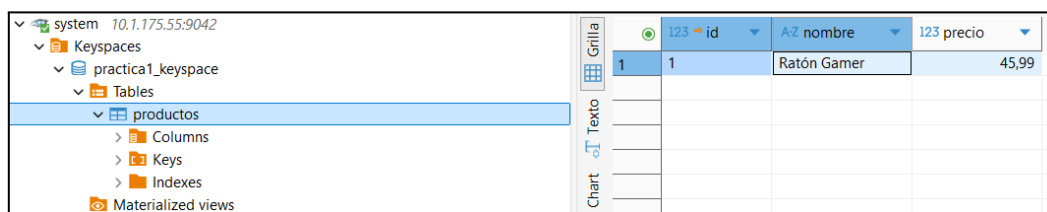
Superada la verificación de red, se procedió a establecer la conexión mediante cliente gráfico (DBeaver) configurado con los siguientes parámetros:

- Host: <IP_DEL_ANFITRION>
- Port: 9042
- Keyspace: practica1_keyspace
- Tabla: productos

La Figura 25 muestra la conexión remota activa y la visualización de la tabla productos, donde se observan los registros insertados previamente en el keyspace practica1_keyspace.

- La correcta visualización de los datos confirma que:
- El servicio Cassandra está operativo.
- El puerto CQL se encuentra correctamente publicado.
- La arquitectura NAT + Port Forwarding permite acceso remoto también en sistemas NoSQL.
- La estructura de datos definida en el keyspace es accesible desde un equipo distinto al servidor.

Este resultado demuestra el cumplimiento del requisito de acceso remoto para Apache Cassandra, evidenciando además las diferencias arquitectónicas respecto a los SGBD relacionales, especialmente en cuanto al modelo de datos y al mecanismo de conexión.



	123 id	Az nombre	123 precio
1	1	Ratón Gamer	45,99

Figura 25. Visualización de la tabla productos en remoto (DBeaver)

8.4. Apache HBase

Para la validación del acceso remoto a Apache HBase, desplegado en modo standalone, se configuró el reenvío de puertos correspondiente a la interfaz web del HBase Master (puerto 16010) en el adaptador NAT de VirtualBox. A diferencia de los SGBD relacionales y de Apache Cassandra, HBase en modo standalone no expone un servicio SQL o CQL tradicional. En este entorno, el mecanismo más directo de acceso remoto es la interfaz web de administración del HBase Master, accesible a través del puerto 16010.

En el fichero docker-compose.yml se configuró el mapeo:

```
ports:
  - "16010:16010"
```

De este modo, el puerto 16010 del contenedor queda publicado en la máquina virtual. Dado que la VM opera en modo NAT, fue necesario habilitar el reenvío de puertos en VirtualBox para permitir el acceso desde el sistema anfitrión. Desde un punto de vista de red, la ruta efectiva de conexión es Windows (host) → VirtualBox (NAT + Port Forwarding) → TurnKey Linux (VM) → Docker → contenedor HBase (Web UI del HMaster, puerto 16010).

Una vez aplicada la configuración y reiniciada la máquina virtual, se verificó el acceso desde el navegador del sistema anfitrión mediante:

`http://localhost:16010`

La interfaz web nativa del HBase Master (Figura 26) permitió comprobar que el sistema estaba operativo y en ejecución. En particular, se verificó:

- Versión del entorno: HBase 2.1.2
- Estado del HMaster: activo
- Presencia de un único RegionServer
- Estado general del clúster: Running
- Visualización de las tablas creadas previamente

Esta comprobación evidencia que el contenedor expone correctamente el servicio hacia la VM, que la configuración de red en VirtualBox es funcional y que el clúster en modo standalone se encuentra operativo, siendo visibles desde la interfaz de administración los objetos creados desde la consola (p. ej., namespace y tabla).

En relación con las herramientas de administración, se evaluó la posibilidad de administrar HBase desde un cliente gráfico universal como DBeaver. Sin embargo, en un despliegue estándar HBase no expone una interfaz SQL nativa, por lo que su administración desde

herramientas orientadas a SQL suele requerir capas adicionales como Apache Phoenix, que proporciona una abstracción SQL sobre HBase [4, 5], o bien drivers ODBC/JDBC de terceros y middleware de conexión [6, 7]. A pesar de que estas opciones existen, se decidió no incorporarlas en el entorno de prácticas por criterios técnicos y de adecuación al objetivo del trabajo. Phoenix no forma parte del despliegue nativo de HBase y su incorporación exige instalar y configurar componentes adicionales, así como cuidar la compatibilidad de versiones. Los drivers de terceros introducen una capa intermedia que traduce consultas SQL al modelo NoSQL, aumentando la dependencia de herramientas no nativas y diluyendo el análisis del modelo propio de HBase. Además, en la arquitectura actual (HBase standalone en Docker bajo NAT), habilitar conectividad externa mediante estos enfoques suele implicar publicar servicios adicionales y abrir nuevos puertos, incrementando la complejidad de red sin aportar un valor conceptual proporcional en el marco de la práctica.

Por ello, para las verificaciones y pruebas se optó por utilizar exclusivamente herramientas nativas. Se empleó HBase Shell para operar sobre el modelo basado en familias de columnas y row keys, y la interfaz web del Master en el puerto 16010 para monitorizar el estado del sistema y validar el acceso remoto, manteniendo el despliegue alineado con el objetivo académico de comprender y utilizar el funcionamiento propio de HBase sin forzar un paradigma relacional sobre el sistema.

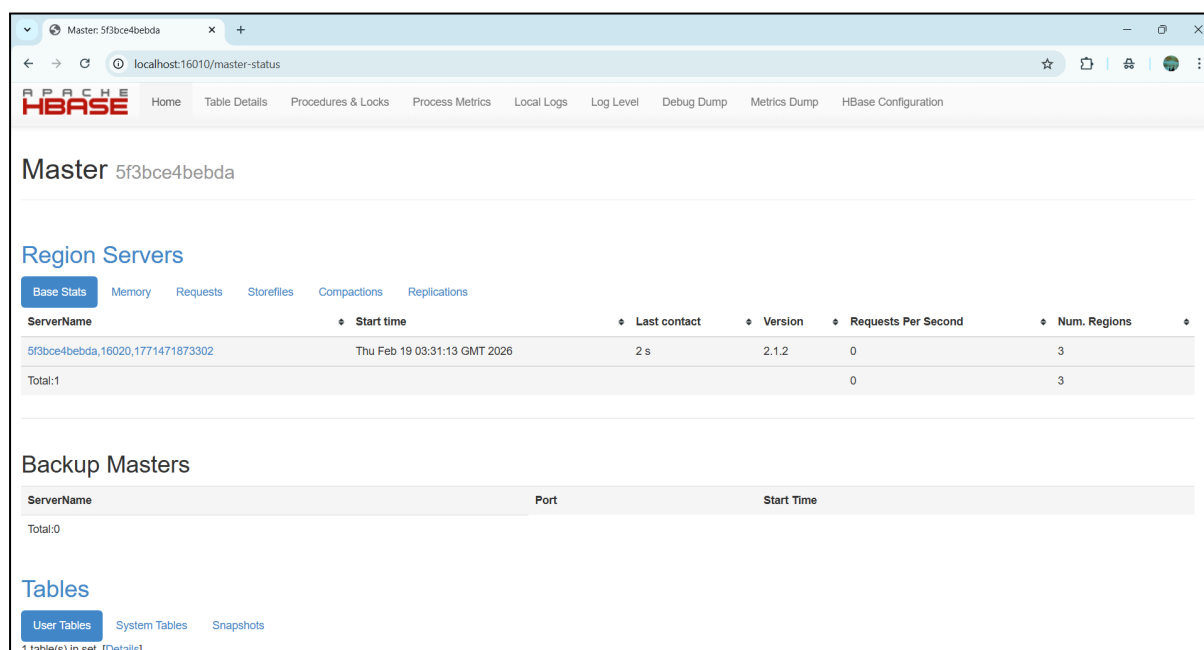


Figura 26. Interfaz web nativa de Apache HBase (URL local)

Posteriormente, con el objetivo de validar el requisito de acceso remoto desde un equipo físico distinto, se comprobó la accesibilidad del puerto 16010 desde un segundo equipo conectado a la misma red Wi-Fi mediante:

```
Test-NetConnection <IP_DEL_ANFITRION> -Port 16010
```

Obteniéndose como resultado:

```
TcpTestSucceeded : True
```

Posteriormente, se accedió mediante navegador web a la siguiente dirección:

`http://<IP_DEL_ANFITRION>:16010`

La correcta carga de la interfaz del HBase Master confirmó que:

- El servicio HBase estaba activo.
- El puerto 16010 se encontraba correctamente publicado en Docker.
- El reenvío de puertos en VirtualBox funcionaba adecuadamente.
- No existían bloqueos a nivel de firewall.

La Figura 27 muestra la interfaz web accesible desde el segundo equipo físico, evidenciando el estado del servidor y las tablas creadas previamente durante la configuración del entorno.

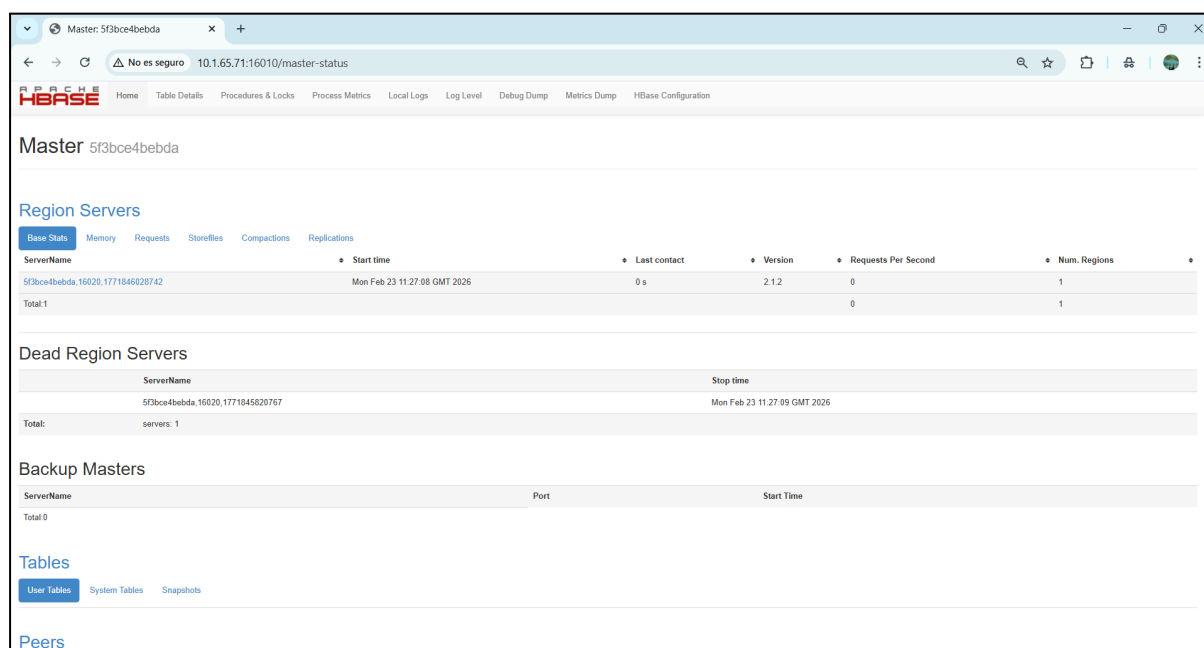


Figura 27. Interfaz web nativa de Apache HBase (URL remota)

Es importante destacar que, en modo standalone, HBase no implementa autenticación ni control de usuarios por defecto, por lo que el acceso a la interfaz web se realiza sin credenciales adicionales dentro de la red local. Este comportamiento difiere significativamente del modelo de seguridad presente en los SGBD relacionales utilizados en la práctica.

La validación realizada confirma el cumplimiento del requisito de acceso remoto para Apache HBase, teniendo en cuenta las particularidades arquitectónicas propias de este sistema NoSQL orientado a almacenamiento distribuido por columnas.

8.5. IBM Db2

Una vez configurada la base de datos y definidos los mecanismos de seguridad internos, se procedió a validar la accesibilidad del servicio IBM Db2 mediante dos evidencias complementarias: (i) conexión de administración mediante DBeaver desde el equipo anfitrión (Windows) y (ii) verificación de accesibilidad desde un segundo equipo físico conectado a la misma red local.

En DB2 LUW (Linux/Unix/Windows), el puerto por defecto para conexiones cliente es el 50000. En el fichero docker-compose.yml definido en el apartado 7.1 se configuró la exposición de dicho puerto mediante el siguiente mapeo:

```
ports:
  - "50000:50000"
```

Este mapeo permite que el puerto 50000 de la máquina virtual (TurnKey Linux) quede redirigido al puerto 50000 del contenedor Docker, donde el motor Db2 escucha las conexiones entrantes. No obstante, dado que la máquina virtual está configurada en modo NAT, los servicios del sistema invitado no son accesibles directamente desde el anfitrión. Por ello fue necesario configurar un reenvío de puertos en VirtualBox. Con la máquina virtual apagada, se accedió a **Configuración** → **Red** → **Adaptador 1 (NAT)** → **Reenvío de puertos** y se añadió una regla TCP asociando 127.0.0.1:50000 en el host con 50000 en el invitado. Tras reiniciar la VM, cualquier conexión dirigida a localhost:50000 desde Windows pasa a ser redirigida hacia el servicio Db2 del contenedor. Desde el punto de vista de arquitectura, la cadena de comunicación queda: Windows (DBeaver) → VirtualBox (NAT + Port Forwarding) → TurnKey Linux (VM) → Docker → contenedor IBM Db2 (50000).

A continuación, se creó en DBeaver una nueva conexión utilizando el driver DB2 LUW, con los parámetros mostrados en la Figura 28. Para la conexión desde el anfitrión se emplearon:

- Host: localhost
- Port: 50000
- Database: PRACTDB2
- User: editor
- Password: editor123

La conexión se estableció correctamente, permitiendo comprobar el correcto funcionamiento del servicio Db2, la adecuada exposición del puerto mediante Docker, la validez de la configuración de red en VirtualBox y la corrección de credenciales y permisos previamente asignados.

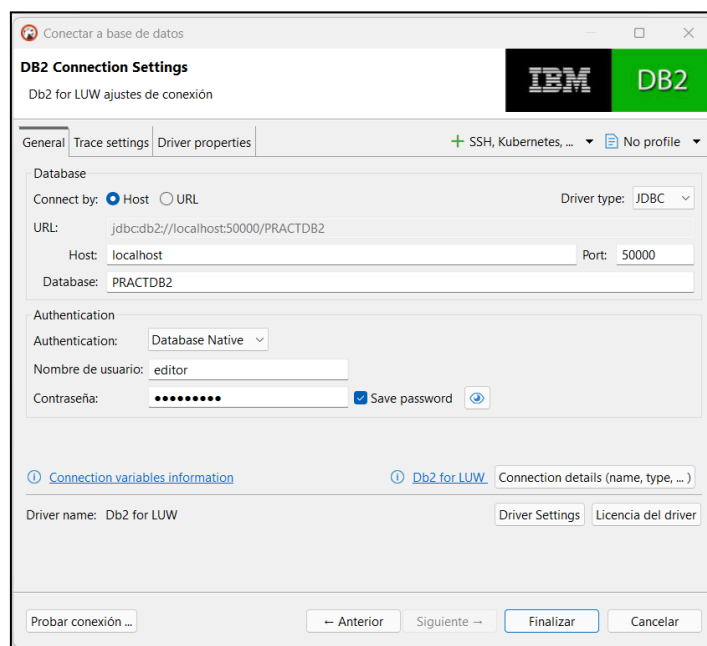


Figura 28. Configuración de la nueva base de datos IBM Db2

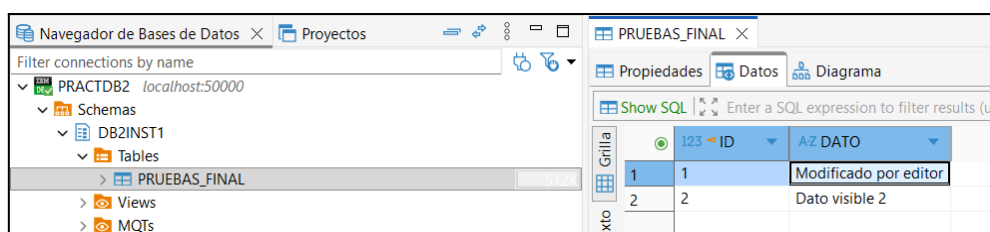


Figura 29. Visualización de la tabla PRUEBAS_FINAL en local (DBeaver)

Posteriormente, con el objetivo de verificar el requisito de acceso remoto desde un equipo físico distinto, se comprobó la accesibilidad del puerto 50000 desde un segundo equipo conectado a la misma red Wi-Fi. Desde dicho equipo se ejecutó en PowerShell:

```
Test-NetConnection <IP_DEL_ANFITRION> -Port 50000
```

Obteniéndose como resultado:

```
TcpTestSucceeded : True
```

Este resultado confirma que:

- El puerto 50000 está accesible desde el exterior.
- El reenvío de puertos configurado en VirtualBox funciona correctamente.
- No existen restricciones de firewall que bloqueen la comunicación.
- La red permite tráfico entre ambos dispositivos.

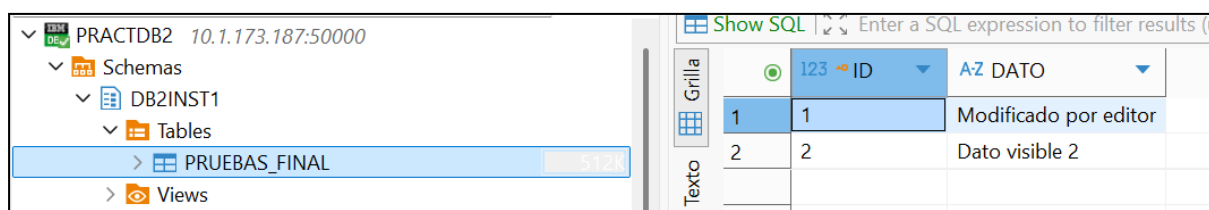
Superada la validación a nivel de red, se estableció conexión mediante cliente gráfico (DBeaver), utilizando los siguientes parámetros:

- Host: <IP_DEL_ANFITRION>
- Port: 50000
- Base de datos: PRACTDB2
- Usuario: editor
- Contraseña: editor123

La Figura 30 muestra la conexión remota activa a la base de datos PRACTDB2, así como la visualización de la tabla PRUEBAS_FINAL dentro del esquema DB2INST1. En la captura puede observarse la correcta recuperación de los registros almacenados, lo que confirma que:

- El servicio DB2 está operativo.
- La base de datos se encuentra accesible desde un equipo distinto al servidor.
- El sistema de autenticación funciona correctamente en escenario distribuido.
- La arquitectura NAT + Port Forwarding es válida también para SGBD relacionales de tipo empresarial.

La correcta ejecución de consultas y visualización de datos desde el segundo equipo físico valida el cumplimiento del requisito de acceso remoto para IBM Db2, completando así la verificación para todos los SGBD instalados en la práctica.



ID	DATO
1	Modificado por editor
2	Dato visible 2

Figura 30. Visualización de la tabla PRUEBAS_FINAL en remoto (DBeaver)

9. Comentarios acerca de las licencias

Para comprender la viabilidad de implementar un Sistema Gestor de Base de Datos en un entorno real o académico, es fundamental analizar los términos legales bajo los que se distribuyen. A continuación, se detallan las licencias de los SGBD utilizados en esta práctica, sus condiciones de uso y sus principales limitaciones.

9.1. PostgreSQL

Licencia: PostgreSQL License (Aprobada por la OSI - Open Source Initiative).

Análisis y limitaciones: Es una licencia de código abierto sumamente permisiva, similar a las licencias BSD o MIT. Permite a cualquier usuario utilizar, copiar, modificar y distribuir el software, ya sea en formato de código fuente o binario, para cualquier propósito, incluido el comercial.

Reflexión: Su principal ventaja es que no impone restricciones de *copyleft* estricto como la GPL, es decir, si una empresa modifica PostgreSQL para crear un producto propio, no está obligada a liberar ese código modificado. La única obligación es mantener el aviso de derechos de autor. No tiene ninguna limitación de hardware ni de volumen de datos.

Fuente: [PostgreSQL Official License](https://www.postgresql.org/licenses/)

9.2. Oracle XE

Licencia: Licencia propietaria.

Análisis y limitaciones: A diferencia de la versión *Enterprise*, Oracle XE es gratuita, pero es software propietario (cerrado). Permite su uso para desarrollo, prototipado e incluso producción, pero impone restricciones técnicas severas por hardware o máquina virtual.

Reflexión: Las limitaciones son estrictas: un máximo de 2 CPUs, 2 GB de memoria RAM y 12 GB de datos de usuario. Además, la licencia prohíbe explícitamente utilizar esta versión para proporcionar servicios de hosting. Por tanto, es excelente para la docencia o pequeñas aplicaciones, pero inviable para entornos empresariales escalables, obligando a migrar a una licencia comercial de pago si el proyecto crece.

Fuente: [Oracle Database Express Edition Licensing](#)

9.3. Apache Cassandra y Apache HBase

Licencia: Apache License, Version 2.0.

Análisis y limitaciones: Ambos SGBD del ecosistema Big Data se encuentran bajo la Fundación Apache. Es una licencia de software libre permisiva que, al igual que la de PostgreSQL, permite el uso comercial, la modificación y la distribución del código.

Reflexión: La gran diferencia y ventaja de la Licencia Apache 2.0 en entornos corporativos es que incluye una concesión expresa de derechos de patentes. Esto protege a los usuarios de posibles demandas por infracción de patentes por parte de los contribuyentes del código. No tiene ninguna limitación en cuanto al número de nodos del clúster o volumen de información, lo que garantiza su viabilidad gratuita para proyectos masivos.

Fuente: [Apache License 2.0](#)

9.4. IBM Db2

Licencia: Licencia propietaria.

Análisis y limitaciones: Al igual que Oracle XE, la versión Community de Db2 es un software de uso gratuito pero de código cerrado, orientado a que los desarrolladores se familiaricen con el ecosistema de IBM.

Reflexión: Aunque es gratuita para entornos de desarrollo e incluso despliegues de producción ligeros, tiene barreras arquitectónicas definidas en la licencia: está limitada a un máximo de 4 núcleos (cores), 16 GB de memoria RAM y 100 GB de almacenamiento por base de datos. Además, la licencia "Non-Warranted" significa que IBM no proporciona soporte técnico oficial ni garantías legales sobre su uso gratuito.

Fuente: [IBM Db2 Edition Comparison and Licensing](#)

10. Dificultades encontradas y soluciones

10.1. PostgreSQL: Error en conexión remota inicial desde DBeaver

Una de las primeras dificultades surgió al intentar conectarse por primera vez a un SGBD desplegado dentro de la máquina virtual desde el sistema anfitrión (Windows) utilizando DBeaver. Durante el primer intento de conexión a PostgreSQL se obtuvo el siguiente error:

- Connection refused: getsockopt
- Connection to localhost:5432 refused

Este resultado indicaba que el servicio no era accesible desde el anfitrión. La causa se encontraba en la configuración de red de la máquina virtual, establecida en modo **NAT**. En este modo, VirtualBox crea una red interna privada para el sistema invitado, por lo que los servicios que escuchan en la VM **no quedan expuestos automáticamente** hacia el host. Por ello, para que una aplicación del anfitrión pueda acceder a un puerto del invitado es necesario definir explícitamente un **reenvío de puertos** (*Port Forwarding*).

Para resolver la incidencia se aplicó el procedimiento siguiente:

1. Apagar la máquina virtual, porque la configuración de red forma parte de los parámetros estructurales de la VM y VirtualBox requiere reinicializar el sistema de red durante el arranque para aplicar correctamente cambios en reglas NAT.
2. Acceder a la configuración de red en VirtualBox:
 - Configuración → Red → Adaptador 1 (NAT) → Reenvío de puertos
3. Añadir una regla de reenvío para PostgreSQL con los valores:
 - Protocolo: TCP
 - IP anfitrión: 127.0.0.1
 - Puerto anfitrión: 5432
 - Puerto invitado: 5432
4. Reiniciar la máquina virtual y repetir la conexión desde DBeaver.

Tras aplicar esta regla, la conexión se estableció correctamente. El flujo de conexión es:

- DBeaver (Windows) → localhost:5432
- VirtualBox NAT (*port forwarding*) → VM TurnKey Linux (5432)
- Docker → PostgreSQL (5432)

De este modo, el puerto **5432** del sistema anfitrión queda vinculado al puerto **5432** del sistema invitado, permitiendo el acceso remoto desde herramientas externas. Este mismo principio se aplicó posteriormente al resto de SGBD que debían exponerse hacia el host.

10.2. Oracle XE: Conflicto conexión remota con imagen “slim”

El problema surgió durante la fase de selección de la imagen de Docker. Con el objetivo de minimizar el consumo de recursos de la máquina virtual, se optó inicialmente por utilizar la versión reducida de Oracle, quedando el fichero *docker-compose.yml* del siguiente modo:

```
version: '3.8'
services:
  oracle:
    image: gvenzl/oracle-xe:21-slim
    container_name: oracle-practical1
    restart: always
    environment:
      ORACLE_PASSWORD: alumno123
      APP_USER: alumno
      APP_USER_PASSWORD: alumno123
    ports:
      - "1521:1521"
    volumes:
      - oracle_data:/opt/oracle/oradata

volumes:
  oracle_data:
```

El proceso de instalación e inicialización del contenedor concluyó correctamente en la terminal. Sin embargo, al intentar realizar la conexión remota desde el sistema anfitrión utilizando DBeaver, el SGBD rechazaba sistemáticamente la conexión devolviendo un error de credenciales inválidas (ORA-01017), a pesar de que los parámetros introducidos coincidían exactamente con las variables APP_USER y APP_USER_PASSWORD.

Análisis y causa: Para aislar el problema, se verificó la conectividad de red configurando un túnel SSH en DBeaver hacia la máquina virtual. El túnel se estableció con éxito, lo que demostraba que el problema no era de red, sino del propio motor de base de datos. La causa técnica reside en que la imagen slim recorta numerosos componentes internos y scripts de inicialización para reducir su tamaño. Esto provoca que, en ciertas ocasiones, la base de datos enchufable (XEPMDB1) no se monte correctamente en el listener externo o que el usuario de la aplicación (APP_USER) no reciba los permisos de sesión remota de forma adecuada durante el primer arranque.

Solución aplicada: Tras diagnosticar que el fallo era inherente a la versión recortada de la imagen, se procedió a modificar el *docker-compose.yml* para descargar y utilizar la imagen oficial completa de Oracle 21c XE. Tras levantar esta nueva imagen, DBeaver autenticó las credenciales y conectó con la base de datos a la primera sin ningún tipo de problema.

10.3. Apache HBase: herramientas de acceso y visualización

A diferencia de los SGBD relacionales utilizados en la práctica, el caso de Apache HBase planteó una dificultad adicional relacionada con la elección de la herramienta de acceso y visualización de datos.

En los sistemas relacionales (PostgreSQL, Oracle XE e IBM Db2), así como en Apache Cassandra, se utilizó DBeaver como interfaz gráfica común para realizar consultas y visualizar tablas. Sin embargo, HBase no está diseñado como un sistema SQL tradicional, sino como una base de datos NoSQL orientada a almacenamiento distribuido por columnas, cuya interacción habitual se realiza mediante [1]:

- HBase Shell (CLI nativa).
- API Java.
- REST (si está habilitado).
- Interfaz web del HBase Master (puerto 16010).

Durante el análisis se estudiaron las siguientes alternativas:

1. Utilizar exclusivamente la HBase Shell (CLI)

Es la herramienta nativa del sistema y permite:

- Creación de tablas.
- Inserción y consulta de datos.
- Operaciones administrativas.

Ventajas:

- Uso directo de comandos propios de HBase.
- Coherencia con la filosofía del sistema.
- No requiere componentes adicionales.

Inconvenientes:

- Menor comodidad en visualización de grandes volúmenes de datos.
- Ausencia de interfaz gráfica homogénea respecto al resto de SGBD.

2. Utilizar la interfaz web del HBase Master

La interfaz accesible mediante `http://<IP>:16010` permite:

- Consultar estado del clúster.
- Visualizar tablas existentes.
- Monitorizar regiones y servidores.

Sin embargo, se comprobó que dicha interfaz no permite:

- Visualización directa del contenido de las tablas.
- Ejecución de consultas sobre los datos almacenados.

Por tanto, su utilidad es fundamentalmente administrativa y no de explotación de datos.

3. Integración mediante Apache Phoenix y DBeaver

Se valoró la posibilidad de emplear Apache Phoenix como capa intermedia que proporciona una interfaz SQL sobre HBase [5]. Phoenix permite:

- Traducir consultas SQL a operaciones nativas de HBase.
- Conectarse desde herramientas como DBeaver mediante driver JDBC.
- Unificar el acceso con el resto de SGBD.

Ventajas:

- Homogeneidad en el uso de DBeaver.
- Posibilidad de consultas SQL estructuradas.
- Mayor comodidad en visualización.

Inconvenientes:

- Introduce una capa adicional no requerida explícitamente en el enunciado.
- Actúa como intermediario de transformación, no como mecanismo nativo.
- Aumenta la complejidad del despliegue.

Tras evaluar las alternativas, se decidió no incorporar inicialmente Apache Phoenix ni integrar HBase en DBeaver, con el objetivo de evitar complejidad adicional innecesaria en esta fase de la práctica. Dado que el uso de interfaz gráfica para HBase no era imprescindible en los objetivos actuales, se optó por mantener el uso de la HBase Shell (CLI) como mecanismo principal de interacción. No obstante, se deja abierta la posibilidad de integrar Phoenix en una fase posterior si se requiere homogeneizar completamente la experiencia de acceso entre los distintos SGBD.

10.4. Complejidades en la configuración de red

Uno de los aspectos que presentó mayor complejidad durante el desarrollo de la práctica fue la habilitación del acceso remoto a los distintos SGBD desplegados en la máquina virtual. Aunque el enunciado exige que los SGBD permitan conexión desde un equipo físico distinto al que aloja el servidor, la arquitectura utilizada (máquina virtual + Docker) introduce varias capas de virtualización que afectan directamente a la conectividad de red.

Inicialmente, la máquina virtual se encontraba configurada en modo NAT (*Network Address Translation*), configuración por defecto en VirtualBox. Este modo permite que la máquina invitada tenga acceso a Internet a través del anfitrión, pero impide que otros dispositivos de la red local puedan establecer conexiones directas hacia ella. En este escenario, aunque los contenedores Docker publicasen correctamente sus puertos, la máquina virtual no era visible como nodo independiente dentro de la red Wi-Fi.

Ante esta situación, se analizaron distintas alternativas:

- **Mantener el modo NAT y configurar redirección de puertos en VBox.**
Esta opción implicaba definir manualmente reglas de reenvío para cada servicio expuesto (5432, 1521, 50000, 9042, 16010, etc.). Aunque inicialmente se consideró que aumentaba la carga de configuración y podía dificultar la escalabilidad, ofrecía

un control explícito sobre los puertos publicados y, en la práctica, resultó ser la alternativa más estable en entorno Wi-Fi, por lo que acabó adoptándose como solución final.

- **Abrir puertos en el router doméstico.**

Esta alternativa fue descartada por implicar la exposición directa de servicios de bases de datos a Internet, introduciendo riesgos de seguridad innecesarios y dependencias sobre la configuración del router doméstico.

- **Utilizar una red privada virtual (VPN).**

Se valoró la posibilidad de emplear soluciones de red virtual superpuesta, como Tailscale o ZeroTier, para simular una red compartida entre equipos ubicados en distintos entornos físicos. Aunque técnicamente viable y segura, se consideró una solución sobredimensionada para el objetivo de la práctica cuando se dispone de red física compartida.

Finalmente, se optó inicialmente por reconfigurar la máquina virtual en modo Adaptador Puente (*Bridged Adapter*). Esta decisión permite que la máquina virtual obtenga una dirección IP propia dentro de la red Wi-Fi, comportándose como un dispositivo adicional en la red local. De este modo, los servicios publicados por los contenedores Docker quedan accesibles directamente desde cualquier equipo conectado a la misma red física, simplificando significativamente la arquitectura de conectividad. Esta solución presentaba varias ventajas: no requería modificaciones en el router, no exponía servicios fuera del entorno de red local, permitía una depuración más sencilla de problemas de conectividad y se alineaba con un entorno controlado y reproducible.

Sin embargo, durante las pruebas se detectó un segundo problema: en entorno portátil con conexión inalámbrica, el Adaptador Puente no obtenía correctamente dirección IPv4 mediante DHCP. La máquina virtual permanecía únicamente con dirección IPv6 local (link-local), impidiendo la comunicación desde otros dispositivos de la red.

Tras identificar que el problema no residía en los SGBD ni en Docker, sino en la capa de conectividad proporcionada por VirtualBox sobre Wi-Fi, se adoptó una solución definitiva basada en NAT con reenvío de puertos:

- Mantener el Adaptador 1 en modo NAT.
- Configurar reglas explícitas de reenvío de puertos (*Port Forwarding*) en VirtualBox.
- Verificar que el puerto anfitrión escuchara en 0.0.0.0 y no únicamente en 127.0.0.1.
- Comprobar la conectividad mediante herramientas de diagnóstico (*Test-NetConnection*) antes de realizar las pruebas con los clientes de cada SGBD.

Con esta configuración, el equipo anfitrión actúa como intermediario entre la red local y la máquina virtual, redirigiendo el tráfico entrante hacia los puertos correspondientes de cada servicio. La solución basada en NAT con reenvío de puertos resultó más estable en entorno Wi-Fi que el uso de Adaptador Puente, especialmente en redes institucionales. Además, permitió mantener un entorno controlado sin necesidad de modificar configuraciones externas ni exponer servicios fuera del ámbito local.

11. Conclusiones y comparativa

Tras completar todo el despliegue y configuración de los cinco SGBDs con los que se ha trabajado en esta sesión, se presenta la siguiente comparativa técnica basada en la experiencia práctica:

- **PostgreSQL, Apache Cassandra e IBM Db2:** Ofrecen la experiencia más ágil y equilibrada. Sus imágenes Docker están altamente optimizadas, garantizando un arranque rápido y un consumo de recursos muy contenido. Además, cuentan con una excelente documentación orientada al desarrollador, lo que permite configurarlos y conectarlos a herramientas gráficas como DBeaver de forma nativa y sin complicaciones.
- **Oracle XE:** Resultó ser el motor relacional más complejo y exigente. Su arquitectura ralentiza enormemente el arranque y consume mucha memoria RAM. Al requerir la integración manual de herramientas adicionales como Oracle Instant Client, cualquier fallo en la configuración obliga a realizar procesos de depuración y reinstalación muy tediosos.
- **Apache HBase:** Presentó la curva de aprendizaje más pronunciada y la peor usabilidad. Su despliegue es pesado al depender de servicios adicionales, como Zookeeper. Su documentación técnica omite guías para la conexión de clientes y su interfaz web no permite manipular datos. De hecho, para lograr conectarlo a DBeaver, fue necesario investigar en fuentes externas para descubrir e implementar Apache Phoenix, una capa traductora de peticiones no documentada de forma clara oficialmente.

En conclusión, mientras que **PostgreSQL, Cassandra e IBM Db2** destacan por su accesibilidad y robustez para despliegues ágiles, soluciones masivas como **Oracle** y **HBase**, pese a su potencia para entornos empresariales, presentan una barrera de entrada técnica y documental demasiado alta para configuraciones iniciales.

12. Organización del trabajo

Apartados del trabajo realizados	Lucía	Luna
1. Configuración del entorno base		
Despliegue de la VM y configuración de red	1	1
Instalación de Docker, Docker Compose y DBeaver	0,5	0,5
2. Implementación de los sistemas gestores de bases de datos (SGBD)		
PostgreSQL: Despliegue, seguridad (RBAC), generación de datos y copias de seguridad	2	2
Oracle XE: Despliegue, seguridad (RBAC), generación de datos y copias de seguridad	5	2
Apache Cassandra: Despliegue, seguridad (RBAC), generación de datos y copias de seguridad	2	1
Apache HBase: Despliegue, seguridad (RBAC), generación de datos y copias de seguridad	1	3
IBM Db2: Despliegue, seguridad (RBAC), generación de datos y copias de seguridad	1	2
3. Conectividad y acceso remoto		
Configuración y pruebas de acceso remoto (DBeaver) unificado	3,5	3,5
4. Investigación y documentación		
Investigación sobre los tipos de licencias de cada SGBD	1	1
Recopilación de dificultades encontradas y soluciones	1	2
Redacción, formato general de la memoria y revisión final	14	14
Preparación de <i>scripts</i>	2	2
	34	34

13. Bibliografía

- [1] The Apache Software Foundation. (s.f.). *Apache HBase™ reference guide*. Recuperado el 23 de febrero de 2026, de <https://hbase.apache.org/book.html>
- [2] 16.2. *SASL Authentication with ZooKeeper*. (s. f.).
<https://svn.apache.org/repos/asf/hbase/hbase.apache.org/trunk/0.94/book/zk.sasl.auth.html>
- [3] *Chapter 1. Secure Apache HBase (TM)*. (s. f.).
<https://svn.apache.org/repos/asf/hbase/hbase.apache.org/trunk/0.94/security.html>
- [4] Cloudera. (s.f.). *Phoenix overview*. Cloudera Documentation. Recuperado el 23 de febrero de 2026, de
<https://docs.cloudera.com/runtime/7.3.1/opdb-overview/topics/phoenix-overview.html>
- [5] The Apache Software Foundation. (s.f.). *Apache Phoenix*. Recuperado el 23 de febrero de 2026, de <https://phoenix.apache.org/>
- [6] DBeaver. (2025, 8 diciembre). *DBeaver PRO | List of All Supported Databases*.
<https://dbeaver.com/databases/>
- [7] CData Software. (s.f.). *Connect to Apache HBase in DBeaver* [o el título exacto que aparezca en la cabecera de la página]. Recuperado el 23 de febrero de 2026, de
<https://www.cdata.com/kb/tech/hbase-odbc-dbeaver.rst>
- [8] *Mockaroo - Random Data Generator and API Mocking Tool | JSON / CSV / SQL / Excel*. (s. f.). <https://www.mockaroo.com/>
- [9] *Welcome to Faker's documentation! — Faker 40.4.0 documentation*. (s. f.).
<https://faker.readthedocs.io/en/master/>

14. Anexo. Scripts de despliegue y configuración

14.1. Anexo I. PostgreSQL

01_docker-compose.yml

```
version: '3.8'
services:
  postgres:
    image: postgres:15
    container_name: postgres-practica1
    restart: always
    environment:
      POSTGRES_USER: alumno          # Superusuario inicial
      POSTGRES_PASSWORD: alumno123
      POSTGRES_DB: practica1_db
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data # Persistencia de datos

volumes:
  pg_data:
```

02_seguridad_y_rols.sql

```
-- 0. Limpieza previa para evitar errores de roles/usuarios ya existentes
DO $$
BEGIN
  -- Borramos los usuarios primero (y revocamos sus dependencias)
  IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'consultor') THEN
    DROP OWNED BY consultor;
    DROP ROLE consultor;
  END IF;

  IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'editor') THEN
    DROP OWNED BY editor;
    DROP ROLE editor;
  END IF;

  -- Luego borramos los roles (y revocamos sus dependencias)
  IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'rol_escritura') THEN
    DROP OWNED BY rol_escritura;
    DROP ROLE rol_escritura;
  END IF;

  IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'rol_lectura') THEN
    DROP OWNED BY rol_lectura;
    DROP ROLE rol_lectura;
  END IF;
END;
```

```
        END IF;
END
$$;

-- 1. Creación de roles
CREATE ROLE rol_lectura;
CREATE ROLE rol_escritura;

-- 2. Asignación de permisos a los roles --> permisos de lectura (base)
GRANT CONNECT ON DATABASE practica1_db TO rol_lectura;
GRANT USAGE ON SCHEMA public TO rol_lectura;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rol_lectura;

-- Permisos de escritura (Hereda de lectura y añade modificaciones)
GRANT rol_lectura TO rol_escritura;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO rol_escritura;

-- Permiso para usar contadores automáticos (SERIAL/SEQUENCES)
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO rol_escritura;

-- 3. Creación y asignación de usuarios
-- consultor solo lectura, mientras que editor tanto lectura como escritura
CREATE USER consultor WITH PASSWORD 'consultor123';
GRANT rol_lectura TO consultor;

CREATE USER editor WITH PASSWORD 'editor123';
GRANT rol_escritura TO editor;
```

03_esquema_y_datos.sql

```
-- 0. Limpieza previa para que el script sea reproducible
DROP TABLE IF EXISTS prestamos CASCADE;
DROP TABLE IF EXISTS lectores CASCADE;
DROP TABLE IF EXISTS libros CASCADE;

-- 1. Definición del esquema relacional
CREATE TABLE lectores (
    dni VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(50),
    email VARCHAR(50)
);

CREATE TABLE libros (
    isbn VARCHAR(20) PRIMARY KEY,
    titulo VARCHAR(100),
    autor VARCHAR(50)
);

CREATE TABLE prestamos (
    id SERIAL PRIMARY KEY,
```

```
dni_lector VARCHAR(15) REFERENCES lectores(dni),
isbn_libro VARCHAR(20) REFERENCES libros(isbn),
fecha_prestamo DATE
);

-- 2. Inserción de datos (Simulando la salida de un script de Mockaroo)
INSERT INTO lectores VALUES ('12345678A', 'Laura Gómez', 'laura@email.com');
INSERT INTO libros VALUES ('978-84-376-0494-7', 'Cien años de soledad', 'Gabriel
García Márquez');
INSERT INTO prestamos (dni_lector, isbn_libro, fecha_prestamo)
VALUES ('12345678A', '978-84-376-0494-7', '2026-02-20');

-- 3. Consulta de prueba (Verificación de relaciones mediante JOIN)
SELECT l.nombre AS Lector, b.titulo AS Libro, p.fecha_prestamo
FROM prestamos p
JOIN lectores l ON p.dni_lector = l.dni
JOIN libros b ON p.isbn_libro = b.isbn;
```

14.2. Anexo II. Oracle

01_docker-compose.yml

```
version: '3.8'
networks:
  bd2_network:
    driver: bridge
services:
  oracle:
    image: container-registry.oracle.com/database/express:latest
    container_name: oracle-xe
    networks:
      - bd2_network
    environment:
      - ORACLE_PWD=oracle123
    ports:
      - "1521:1521"
      - "5500:5500"
    volumes:
      - oracle_data:/opt/oracle/oradata

volumes:
  oracle_data:
```

02_seguridad_y_roles.sql

```
-- Cambio a la base de datos enchufable (Pluggable Database)
ALTER SESSION SET CONTAINER = XEPDB1;

-- 0. Limpieza previa (Ignora errores si los usuarios/roles no existen aún)
```

```
BEGIN EXECUTE IMMEDIATE 'DROP USER alumno CASCADE'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP USER editor CASCADE'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP USER consultor CASCADE'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP ROLE rol_escritura'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP ROLE rol_lectura'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/

-- 1. Creación de roles
CREATE ROLE rol_lectura;
CREATE ROLE rol_escritura;

-- 2. Asignación de permisos a los roles
GRANT CREATE SESSION TO rol_lectura;
GRANT rol_lectura TO rol_escritura;
GRANT RESOURCE TO rol_escritura;
GRANT CREATE TABLE TO rol_escritura;

-- 3. Creación y asignación de usuarios
CREATE USER alumno IDENTIFIED BY alumno123;
GRANT DBA TO alumno;

CREATE USER editor IDENTIFIED BY editor123;
GRANT rol_escritura TO editor;
ALTER USER editor QUOTA UNLIMITED ON USERS;

CREATE USER consultor IDENTIFIED BY consultor123;
GRANT rol_lectura TO consultor;
```

03_esquema_y_datos.sql

```
-- Cambio a la base de datos enchufable
ALTER SESSION SET CONTAINER = XEPDB1;

-- 0. Limpieza previa de tablas simulando "IF EXISTS" (CASCADE CONSTRAINTS borra
también las dependencias)
BEGIN EXECUTE IMMEDIATE 'DROP TABLE prestamos CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE libros CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP TABLE lectores CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE productos CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/

-- 1. Definición del esquema
CREATE TABLE lectores (dni VARCHAR2(15) PRIMARY KEY, nombre VARCHAR2(50));
CREATE TABLE libros (isbn VARCHAR2(20) PRIMARY KEY, titulo VARCHAR2(100));
CREATE TABLE prestamos (
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    dni_lector VARCHAR2(15) REFERENCES lectores(dni),
    isbn_libro VARCHAR2(20) REFERENCES libros(isbn),
    fecha_prestamo DATE
);
CREATE TABLE productos (id NUMBER, nombre VARCHAR2(50));

-- 2. Inserción de catálogo base
INSERT INTO lectores VALUES ('12345678A', 'Laura Gómez');
INSERT INTO libros VALUES ('978-84', 'Cien años de soledad');
INSERT INTO libros VALUES ('978-99', 'El Quijote');
INSERT INTO productos VALUES (1, 'Monitor 4K');
COMMIT;

-- 3. Estrategia PL/SQL: Generación masiva de 500 préstamos
BEGIN
    FOR i IN 1..500 LOOP
        INSERT INTO prestamos (dni_lector, isbn_libro, fecha_prestamo)
        VALUES (
            '12345678A',
            CASE ROUND(DBMS_RANDOM.VALUE(1,2)) WHEN 1 THEN '978-84' ELSE
'978-99' END,
            SYSDATE - ROUND(DBMS_RANDOM.VALUE(1, 30))
        );
    END LOOP;
    COMMIT;
END;
/

-- 4. Asignación de permisos de lectura para que el consultor pueda hacer
SELECTs
GRANT SELECT ON productos TO rol_lectura;
GRANT SELECT ON lectores TO rol_lectura;
GRANT SELECT ON libros TO rol_lectura;
GRANT SELECT ON prestamos TO rol_lectura;
```

14.3. Anexo III. Apache Cassandra

01_docker-compose.yml

```
version: '3.8'

networks:
  bd2_network:
    driver: bridge

services:
  cassandra:
    image: cassandra:latest
    container_name: cassandra-db
    networks:
      - bd2_network
    ports:
      - "9042:9042" # Puerto estándar de Cassandra (CQL)
    environment:
      - CASSANDRA_CLUSTER_NAME=Practica1_Cluster
    volumes:
      - cassandra_data:/var/lib/cassandra

volumes:
  cassandra_data:
```

02_seguridad_y_rols.cql

```
-- 0. Limpieza previa para garantizar la reproducibilidad
DROP USER IF EXISTS consultor;
DROP USER IF EXISTS editor;
DROP USER IF EXISTS alumno;
DROP ROLE IF EXISTS rol_escritura;
DROP ROLE IF EXISTS rol_lectura;
DROP KEYSPACE IF EXISTS practica1_keyspace;

-- 1. Creación del espacio de nombres (Keyspace)
CREATE KEYSPACE practica1_keyspace
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

-- 2. Creación de Roles Lógicos
CREATE ROLE rol_lectura;
CREATE ROLE rol_escritura;

-- 3. Creación de Usuarios
CREATE USER alumno WITH PASSWORD 'alumno123' SUPERUSER;
CREATE USER editor WITH PASSWORD 'editor123';
CREATE USER consultor WITH PASSWORD 'consultor123';

-- 4. Asignación de Permisos a Los ROLES
```

```
GRANT ALL PERMISSIONS ON KEYSPACE practica1_keyspace TO rol_escritura;
GRANT SELECT ON KEYSPACE practica1_keyspace TO rol_lectura;

-- 5. Asignación de Roles a Los USUARIOS
GRANT rol_escritura TO editor;
GRANT rol_lectura TO consultor;
```

03_esquema_y_datos.cql

```
-- Conectamos al Keyspace de la práctica
USE practica1_keyspace;

-- 0. Limpieza previa de las tablas
DROP TABLE IF EXISTS productos;
DROP TABLE IF EXISTS historial_prestamos;

-- 1. Creación de la tabla para pruebas de seguridad
CREATE TABLE productos (
    id int PRIMARY KEY,
    nombre text,
    precio float
);

-- 2. Creación del esquema desnormalizado (sin relaciones/Foreign Keys)
CREATE TABLE historial_prestamos (
    dni_lector text,
    fecha_prestamo date,
    nombre_lector text,
    titulo_libro text,
    PRIMARY KEY (dni_lector, fecha_prestamo)
);

-- 3. Inserción de datos manuales (DML)
INSERT INTO productos (id, nombre, precio)
VALUES (1, 'Ratón Gamer', 45.99);

INSERT INTO historial_prestamos (dni_lector, fecha_prestamo, nombre_lector,
titulo_libro)
VALUES ('12345678A', '2026-02-20', 'Laura Gómez', 'Cien años de soledad');
```

14.4. Anexo IV. Apache HBase

01_docker-compose.yml

```
services:
  hbase:
    image: dajobe/hbase
    container_name: hbase-db
    restart: unless-stopped
```

```
ports:
  - "16010:16010"    # Web UI del Master
volumes:
  - ./data:/opt/hbase-data
```

02_seguridad_y_roles.txt

```
=====
NOTA TÉCNICA SOBRE LA SEGURIDAD EN APACHE HBASE (MODO STANDALONE)
=====
```

Tal y como se detalla y justifica en la Memoria de la Práctica, en un entorno HBase desplegado en modo 'standalone' mediante Docker (sin integración con Hadoop, HDFS distribuido, ni servidor Kerberos/KDC), no es posible habilitar de forma efectiva el subsistema de autorización (ACLs) ni el control de roles.

Aunque se modifique el fichero 'hbase-site.xml' para incluir las propiedades:

- hbase.security.authentication = simple
- hbase.security.authorization = true
- hbase.coprocessor.master.classes = AccessController

El sistema no dispone de la infraestructura subyacente de autenticación segura (como ZooKeeper con SASL o tickets de Kerberos) requerida por el AccessController de HBase para gestionar usuarios y permisos (comandos GRANT/REVOKE).

Por tanto, este SGBD opera bajo un modelo de confianza total en la red local del contenedor, no siendo viable la creación de los roles 'consultor' y 'editor' requeridos en la práctica mediante la HBase Shell nativa.

03_esquema_y_datos.txt

```
#
=====
# SCRIPT DE INICIALIZACIÓN Y CARGA DE DATOS PARA APACHE HBASE SHELL
# Instrucciones: Pegar estos comandos directamente en la consola interactiva
# a la que se accede mediante: docker exec -it hbase-db hbase shell
#
=====

# 0. Limpieza previa (Simulación de DROP IF EXISTS)
# En HBase es obligatorio deshabilitar una tabla antes de poder borrarla.
disable 'espacioHBase:pruebas_final'
drop 'espacioHBase:pruebas_final'
disable 'historial_prestamos'
drop 'historial_prestamos'
drop_namespace 'espacioHBase'
```

```
# 1. Creación del Namespace y Tablas
create_namespace 'espacioHBase'

# Tabla para pruebas genéricas (Familia de columnas: 'info')
create 'espacioHBase:pruebas_final', 'info'

# Tabla para el modelo desnormalizado de la biblioteca (Familia de columnas:
'info')
create 'historial_prestamos', 'info'

# 2. Inserción de datos (Comandos PUT)
# 2.1 Datos de prueba genéricos
put 'espacioHBase:pruebas_final', '1', 'info:dato', 'Dato visible 1'
put 'espacioHBase:pruebas_final', '2', 'info:dato', 'Dato visible 2'

# 2.2 Datos del modelo de biblioteca (RowKey = DNI-FECHA)
put 'historial_prestamos', '12345678A-20260220', 'info:nombre_lector', 'Laura
Gómez'
put 'historial_prestamos', '12345678A-20260220', 'info:titulo_libro', 'Cien años
de soledad'

put 'historial_prestamos', '12345678A-20260225', 'info:nombre_lector', 'Laura
Gómez'
put 'historial_prestamos', '12345678A-20260225', 'info:titulo_libro', 'El
Quijote'

# 3. Consultas de verificación
# 3.1 Recuperar un dato concreto por su RowKey exacta
get 'espacioHBase:pruebas_final', '1'

# 3.2 Escanear todo el historial de préstamos de un usuario específico usando
ROWPREFIXFILTER
scan 'historial_prestamos', {ROWPREFIXFILTER => '12345678A'}
```

14.5. Anexo V. IBM Db2

01_docker-compose.yml

```
services:
  db2:
    image: ibmcom/db2
    container_name: contenedor_db2
    restart: always
    privileged: true
    ports:
      - "50000:50000"
    environment:
```

```

LICENSE: accept
DB2INST1_PASSWORD: db2inst1
DBNAME: db2inst1
PERSISTENT_HOME: "false"
ARCHIVE_LOGS: "false"
TEXT_SEARCH: "false"
volumes:
- ./data:/database

```

02_seguridad_y_rols.sql

```

--
=====
-- 02_seguridad_y_rols.sql (Db2)  -- Terminador: @
-- Ejecutar con: db2 -td@ -vf 02_seguridad_y_rols.sql
--
=====

CONNECT TO PRACTDB2@

BEGIN
  DECLARE CONTINUE HANDLER FOR SQLSTATE '42704' BEGIN END;
  EXECUTE IMMEDIATE 'DROP ROLE ROL_Lectura';
END
@

BEGIN
  DECLARE CONTINUE HANDLER FOR SQLSTATE '42704' BEGIN END;
  EXECUTE IMMEDIATE 'DROP ROLE ROL_Escritura';
END
@

CREATE ROLE ROL_Lectura
@

CREATE ROLE ROL_Escritura
@

GRANT CONNECT ON DATABASE TO ROLE ROL_Lectura
@

GRANT CONNECT ON DATABASE TO ROLE ROL_Escritura
@

GRANT DATAACCESS ON DATABASE TO ROLE ROL_Escritura
@

GRANT DBADM ON DATABASE TO ROLE ROL_Escritura
@

```

```

REVOKE DATAACCESS ON DATABASE FROM ROLE ROL_LECTURA
@

GRANT ROLE ROL_LECTURA TO USER CONSULTOR
@

GRANT ROLE ROL_ESCRITURA TO USER EDITOR
@

TERMINATE
@

```

03_esquema_y_datos.sql

```

CONNECT TO PRACDDB2;

-- 0. Limpieza previa de tablas para garantizar la reproducibilidad
DROP TABLE IF EXISTS prestamos;
DROP TABLE IF EXISTS libros;
DROP TABLE IF EXISTS lectores;
DROP TABLE IF EXISTS pruebas_final;

-- =====
-- 1. ESQUEMA DE PRUEBAS DE SEGURIDAD
-- =====
CREATE TABLE pruebas_final (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dato VARCHAR(100)
);

INSERT INTO pruebas_final (dato) VALUES ('Dato visible 1');
INSERT INTO pruebas_final (dato) VALUES ('Dato visible 2');

-- Aplicamos la restricción de seguridad descubierta en la práctica:
-- Damos solo permiso SELECT al rol de lectura a nivel de tabla.
GRANT SELECT ON TABLE pruebas_final TO ROLE ROL_LECTURA;

-- =====
-- 2. ESQUEMA DE LA BIBLIOTECA
-- =====
CREATE TABLE lectores (
    dni VARCHAR(15) NOT NULL PRIMARY KEY,
    nombre VARCHAR(50)
);

CREATE TABLE libros (
    isbn VARCHAR(20) NOT NULL PRIMARY KEY,
    titulo VARCHAR(100)
);

```

```
CREATE TABLE prestamos (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    dni_lector VARCHAR(15) REFERENCES lectores(dni),  
    isbn_libro VARCHAR(20) REFERENCES libros(isbn),  
    fecha_prestamo DATE  
);  
  
-- Inserción de catálogo base  
INSERT INTO lectores (dni, nombre) VALUES ('12345678A', 'Laura Gómez');  
INSERT INTO libros (isbn, titulo) VALUES ('978-84', 'Cien años de soledad');  
  
-- Inserción manual de prueba para validar la restricción  
INSERT INTO prestamos (dni_lector, isbn_libro, fecha_prestamo)  
VALUES ('12345678A', '978-84', CURRENT DATE);  
  
-- Concesión de Lectura sobre las tablas de la biblioteca  
GRANT SELECT ON TABLE lectores TO ROLE ROL_LECTURA;  
GRANT SELECT ON TABLE libros TO ROLE ROL_LECTURA;  
GRANT SELECT ON TABLE prestamos TO ROLE ROL_LECTURA;  
  
-- =====  
-- NOTA SOBRE LA CARGA MASIVA (IMPORT):  
-- Para importar los datos generados masivamente, se ejecutaría:  
-- IMPORT FROM prestamos_generados.csv OF DEL INSERT INTO prestamos;  
-- =====  
  
TERMINATE;
```